

Lab-05: GPIO Read-Write Using Keil- STM32CubeMX – HAL Lib

1. Objectives

- ✓ Configure GPIO Output Pin Within CubeMX Tool
- ✓ Use HAL_GPIO_Write To Change The Pin State
- ✓ And Use The HAL_Delay() & Know How It Works

2. What do I need to program the STM32?

2.1. Documentation

Before you really use a specific microcontroller in your project, you should at least glance at the datasheet. You should check what peripherals it offers, how many GPIOs it has, and whether the clock is sufficient for your requirements. But relax, you don't need to read the whole thing, at least not right away

2.2. Language

I mean programming languages, of course. The only correct option here is **C**. Don't bother with other creations. The time will come for that once you get the C language.

2.3. Hardware

For starters, I recommend any **Nucleo** or **Discovery**, evaluation board. There's an ST-LINK programmer integrated into it, so you don't need to buy an external one. Additionally, you can use it later to program other STM32 microcontrollers.

Nucleo boards have an Arduino-compatible connector. So if you have any shield for the Arduino UNO board, you can use it with Nucleo. Thanks to this, you have easy access to various types of expansion boards with sensors or controllers.

Discovery boards, on the other hand, very often it has some sensors built into the board. You can also start your adventure without buying any external boards. You can also choose the recently popular **STM32 BluePill**. It has a nice, small size, however, it doesn't have a built-in programmer, so you have to buy it separately. Any **STLink V2** clone ([like this](#)) will be ok. It cost pennies and you will surely find it locally.

2.4. Software

ST Microelectronics actively supports developers. That's why they made available a wide portfolio of free software. I'm not gonna dwell on all the options such as (paid) **IAR Embedded Workbench**, (paid / free) **µVision – Keil**, or free **SW4STM32 – System Workbench** or **Eclipse** with plugins.

I'm gonna focus on the only right choice in my opinion: **STM32CubeIDE**. IDE stands for Integrated Development Environment.

What is STM32CubeIDE?

STM32CubeIDE is an all-in-one development software ecosystem. It has integrated: a microcontroller selector, peripheral configurator, code generator, compiler, debugger, and programmer. Moreover, it is completely free for both hobby and professional purposes.

It is based on the popular **Eclipse**, so its capabilities can be extended even more with external plugins. However, I didn't need to do that yet.

2.5. Framework

What do I mean by that? An approach to writing code itself. In the sense of whether and which libraries you want to use. You can choose from:

– Pure C language

If you want your code to be optimized as much as possible, take up a minimum amount of memory, and be super fast this option is for you. However, it's by far the most difficult path. In this case, you're working directly on the registers, so Datasheet, Erratum to Datasheet, Application Notes, and Reference Notes you have to get to know cover to cover.

If you are just starting, I strongly advise against this approach.

– Arduino

You've probably heard and maybe even used Arduino before. True, it makes it much easier to get started, however it uses a simplified version of the C / C++ language. So learning it won't let you migrate later on to another IDE, or even to another family of microcontrollers. Arduino is very limited when it comes to STM32. Only supports a few boards, e.g. **BluePill**.

Additionally, a lot of libraries are made by the community, so the quality of them really varies. Believe me, I've seen some really poor written libraries.

This is why I'm not a big fan of Arduino.

– (LL) Low Layers libraries

Low-level libraries integrated with STM32CubeIDE. If you have a limited amount of available memory and your code needs to be well optimized this is the option for you. It requires you to have a fairly good knowledge of documentation and registers. Fortunately, not as much as in the case of "pure C". These libraries are very oriented towards a specific uC model, so it makes migration a bit more difficult.

– HAL (Hardware Abstraction Layer)

These are high-level libraries also integrated with STM32CubeIDE. I think this is the best option (especially if you are just starting). This is what I'm gonna cover in this article.

Admittedly, the LL has more options, but I'm sure that you'll not even come close to the limitations of HAL.

And now, the best part: HAL libraries can coexist with LL libraries! If you want a particular periphery to use LL instead of HAL, you just change the setting in STM32CubeIDE.

3. STM32 HAL Library Tutorial

STMCube® is STMicroelectronics's original initiative to ease and accelerate the development cycle of embedded products. STM32Cube covers the STM32 portfolio of microcontrollers.

It provides the developer with all the low-level drivers, APIs, and enables him/her to dedicate most of the effort to develop the application layer and any required middleware.

4. STM32 HAL Library Drivers

The STM32CubeMX, a graphical software configuration tool that allows generating C initialization code using graphical wizards.

The STM32Cube Hardware Abstraction Layer (HAL), an STM32 abstraction layer embedded software ensuring maximized portability across the STM32 microcontroller. The HAL is available for all the hardware peripherals.

The low-layer APIs (LL) offering a fast light-weight expert-oriented layer that is closer to the hardware than the HAL. The LL APIs are available only for a set of peripherals.

The HAL and LL are complementary and cover a wide range of applications requirements:

- The HAL offers high-level and feature-oriented APIs, with a high-portability level. They hide the MCU and peripheral complexity to end-user.
- The LL offers low-level APIs at registers level, with better optimization but less portability. They require deep knowledge of the MCU and peripherals specifications.

The source code of HAL and LL drivers is developed in Strict ANSI-C which makes it independent from the development tools. It is fully documented and is MISRA-C 2004 compliant.

5. STM32 HAL Drivers Examples

Initialization and de-initialization functions

This section contains the following APIs:

- [`HAL\_GPIO\_Init\(\)`](#)
- [`HAL\_GPIO\_DeInit\(\)`](#)

IO operation functions

This section contains the following APIs:

- [`HAL\_GPIO\_ReadPin\(\)`](#)
- [`HAL\_GPIO\_WritePin\(\)`](#)
- [`HAL\_GPIO\_TogglePin\(\)`](#)
- [`HAL\_GPIO\_LockPin\(\)`](#)
- [`HAL\_GPIO\_EXTI\_IRQHandler\(\)`](#)

HAL Detailed Function Description

HAL_GPIO_Init

Function name	void HAL_GPIO_Init (GPIO_TypeDef * GPIOx, GPIO_InitTypeDef * GPIO_Init)
Function description	Initialize the GPIOx peripheral according to the specified parameters in the GPIO_Init.
Parameters	• GPIOx: where x can be (A..H) to select the GPIO peripheral for STM32L4 family • GPIO_Init: pointer to a GPIO_InitTypeDef structure that contains the configuration information for the specified GPIO peripheral.
Return values	• None:

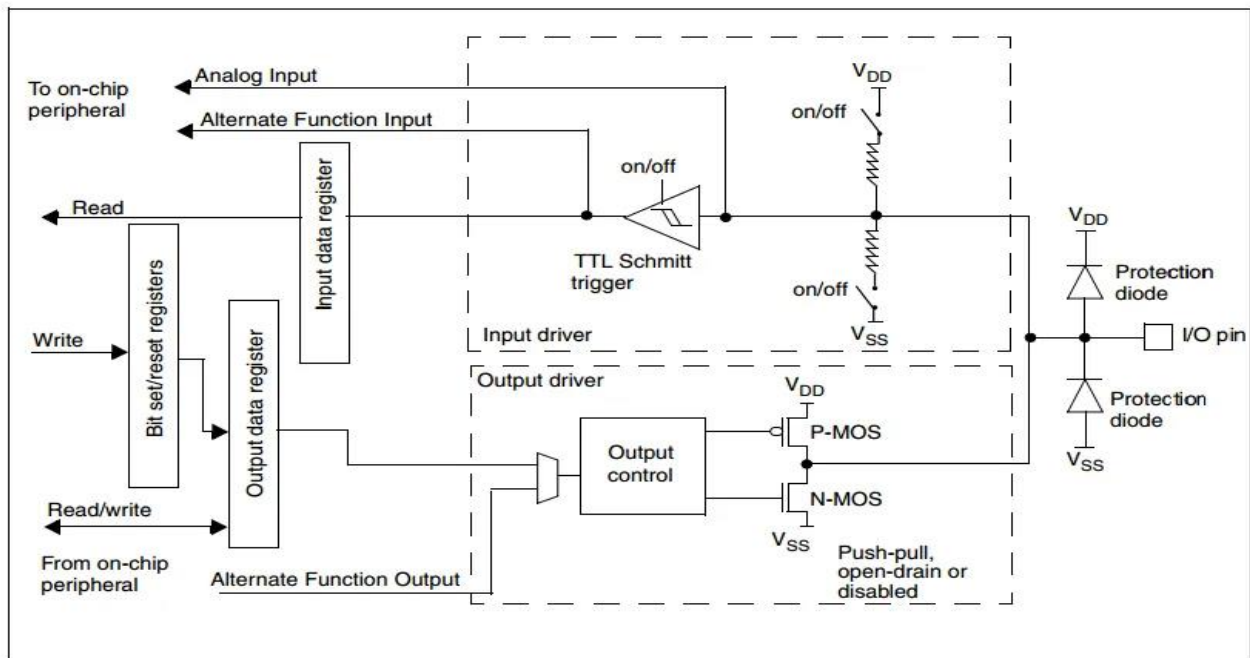
In the same manner, there are low-level hardware drivers for almost all the hardware peripherals in the STM32 microcontrollers. Including Timers, ADC, USART, I2C, USB, DAC, Comparators, etc.

We'll configure these peripherals and generate the initialization C-Code using the CubeMX software tool. Here is a screenshot for what it looks like inside the interface where we configure the timer module for example.

6. STM32 GPIO Ports

Each of the general-purpose I/O ports has two 32-bit configuration registers, two 32-bit data registers, a 32-bit set/reset register, a 16-bit reset register, and a 32-bit locking register. Each I/O port bit is freely programmable, however, the I/O port registers have to be accessed as 32-bit words (half-word or byte accesses are not allowed). The purpose of the set/reset registers is to allow atomic read/modify accesses to any of the GPIO registers. This way, there is no risk that an IRQ occurs between the read and modify access.

Here is a digital diagram for the internal structure of a typical GPIO pin. It shows the diode protection, internal pull-up or down enable/disable, and also the push-pull output driver, output enable/disable for switching between input/output pin modes, Schmitt-triggered digital input, analog input, digital input, analog input.



You have to assume that all GPIO pins are not 5v tolerant by default until you find in the datasheet that a specific pin is 5v tolerant, only then you can use it as a 5v pin. The pins are mostly 3.3v and can be damaged when hooked up to 5v digital input lines. Level shifting may be mandatory in many cases.

Symbol	Ratings	Min	Max	Unit
$V_{DD} - V_{SS}$	External main supply voltage (including V_{DDA} and V_{DD}) ⁽¹⁾	-0.3	4.0	V
$V_{IN}^{(2)}$	Input voltage on five volt tolerant pin	$V_{SS} - 0.3$	$V_{DD} + 4.0$	
	Input voltage on any other pin	$V_{SS} - 0.3$	4.0	

So you have to be careful with the voltage level for input pins. And also you've to pay attention to the output current when you set GPIO output pins. The maximum current that could be sourced or sunk into any GPIO pin is 25mA as per the datasheet. And you've to check it for the specific target microcontroller you're dealing with.

Symbol	Ratings	Max.	Unit
I_{VDD}	Total current into V_{DD}/V_{DDA} power lines (source) ⁽¹⁾	150	mA
I_{VSS}	Total current out of V_{SS} ground lines (sink) ⁽¹⁾	150	
I_{IO}	Output current sunk by any I/O and control pin	25	
	Output current source by any I/Os and control pin	-25	

7. STM32 GPIO Speed

- For Input Mode

When a GPIO pin is set to input mode, the data present on the I/O pin is sampled into the Input Data Register every APB2 clock cycle. This means the APB2 bus speed determines the input sampling speed for the GPIO pins.

- For Output Mode

When a GPIO pin is set to output mode, you'll have the option to configure the pin speed mode by programming the respective bits in the configuration registers. Down below is a table for the different modes available in the datasheet for the STM32F103C8 microcontroller.

Table 20. Port bit configuration table

Configuration mode		CNF1	CNF0	MODE1	MODE0	PxODR register	
General purpose output	Push-pull	0	0	01 10 11 see <i>Table 21</i>		0 or 1	
	Open-drain		1			0 or 1	
Alternate Function output	Push-pull	1	0			Don't care	
	Open-drain		1			Don't care	
Input	Analog	0	0	00		Don't care	
	Input floating		1			Don't care	
	Input pull-down	1	0			0	
	Input pull-up					1	

Table 21. Output MODE bits

MODE[1:0]	Meaning
00	Reserved
01	Maximum output speed 10 MHz
10	Maximum output speed 2 MHz
11	Maximum output speed 50 MHz

However, other microcontrollers will have different options for the pin output speed. For example, the other microcontroller we'll be using in this course STM32L432KC. It has a dedicated speed control register for GPIO pins, which have 4 different speed options: Low-speed, Medium-speed, High-speed, Very High-speed.

Bits 31:0 **OSPEED[15:0][1:0]**: Port x configuration I/O pin y (y = 15 to 0)

These bits are written by software to configure the I/O output speed.

00: Low speed

01: Medium speed

10: High speed

11: Very high speed

You'll find what do these speed grades translate to in the device datasheet io pin characteristics. Here is the table for this parameter from the STM32L432KC datasheet table.

Note: Typical values depend on various parameters of which the V_{DDi0} line voltage, the total pin capacitance.

- Low Speed -> Up to 10MHz
- Medium Speed -> Up to 50MHz
- High Speed -> Up to 100MHz

- Very High Speed -> Up to 180MHz

Table 59. I/O AC characteristics⁽¹⁾⁽²⁾

Speed	Symbol	Parameter	Conditions	Min	Max	Unit
00	Fmax	Maximum frequency	C=50 pF, 2.7 V≤V _{DDIOx} ≤3.6 V	-	5	MHz
			C=50 pF, 1.62 V≤V _{DDIOx} ≤2.7 V	-	1	
			C=50 pF, 1.08 V≤V _{DDIOx} ≤1.62 V	-	0.1	
			C=10 pF, 2.7 V≤V _{DDIOx} ≤3.6 V	-	10	
			C=10 pF, 1.62 V≤V _{DDIOx} ≤2.7 V	-	1.5	
			C=10 pF, 1.08 V≤V _{DDIOx} ≤1.62 V	-	0.1	
	Tr/Tf	Output rise and fall time	C=50 pF, 2.7 V≤V _{DDIOx} ≤3.6 V	-	25	ns
			C=50 pF, 1.62 V≤V _{DDIOx} ≤2.7 V	-	52	
			C=50 pF, 1.08 V≤V _{DDIOx} ≤1.62 V	-	140	
			C=10 pF, 2.7 V≤V _{DDIOx} ≤3.6 V	-	17	
			C=10 pF, 1.62 V≤V _{DDIOx} ≤2.7 V	-	37	
			C=10 pF, 1.08 V≤V _{DDIOx} ≤1.62 V	-	110	

- GPIO Bit Atomic Operations

There is no need for the software to disable interrupts when programming the GPIOx_ODR at bit level: it is possible to modify only one or several bits in a single atomic APB2 write access. This is achieved by programming to '1' the Bit Set/Reset Register (GPIOx_BSRR, or for reset only GPIOx_BRR) to select the bits to modify. The unselected bits will not be modified.

- STM32 GPIO External Interrupts

All ports have external interrupt capability. To use external interrupt lines, the port must be configured in input mode. We'll discuss this in a more detailed manner when we get into the EXTI (external interrupt/event controller) topic.

- Peripheral Pin Select (PPS)

To optimize the number of peripheral I/O functions for different device packages, it is possible to remap some alternate functions to some other pins. This is achieved by software, by programming the corresponding registers.

This option can help you remap the peripherals io pins, so you don't have to change so much in the PCB layout when you change the target microcontroller on the board. This can be extremely advantageous and ease the routing process. And help you move the high-speed signals away so as to reduce the noise level at certain parts.

- STM32 GPIO Locking Mechanism

The locking mechanism allows the IO configuration to be frozen. When the LOCK sequence has been applied on a port bit, it is no longer possible to modify the value of the port bit until the next reset.

- STM32 GPIO Configurations

Subject to the specific hardware characteristics of each I/O port listed in the *datasheet*, each port bit of the General Purpose IO (GPIO) Ports, can be individually configured by software in several modes:

Output Configuration

- Output Open-Drain
- Output Push-Pull

Input Configuration

- Input Floating (Hi-Z)
- Input Pull-Up
- Input Pull-Down

Alternate Function Configuration

- Alternate Function Push-Pull
- Alternate Function Open-Drain

Analog Configuration

- STM32 GPIO Concluding Remarks

Not all GPIO Pin are 5v tolerant. Mostly they are 3.3v.

You've to enable the clock for the GPIO you want to use regardless of the mode of operation.

When configuring a GPIO output pin, you can select the speed suitable for your application. Whether you need high-frequency switching on the IO line or not.

The APB2 bus speed determines the sampling rate for all GPIO input configured pins.

You can disconnect any GPIO pin (make it high impedance) by setting it into input mode with Hi-Z.

You can freeze or lock the GPIO pins configuration after the system initialization if no change is intended using the locking mechanism.

Alternative function pins can be interchanged (remapped) internally so you can enjoy more freedom in routing these signals.

All GPIO pins are interrupt-capable pins but have to be configured within the EXTI as we'll see later on in the course.

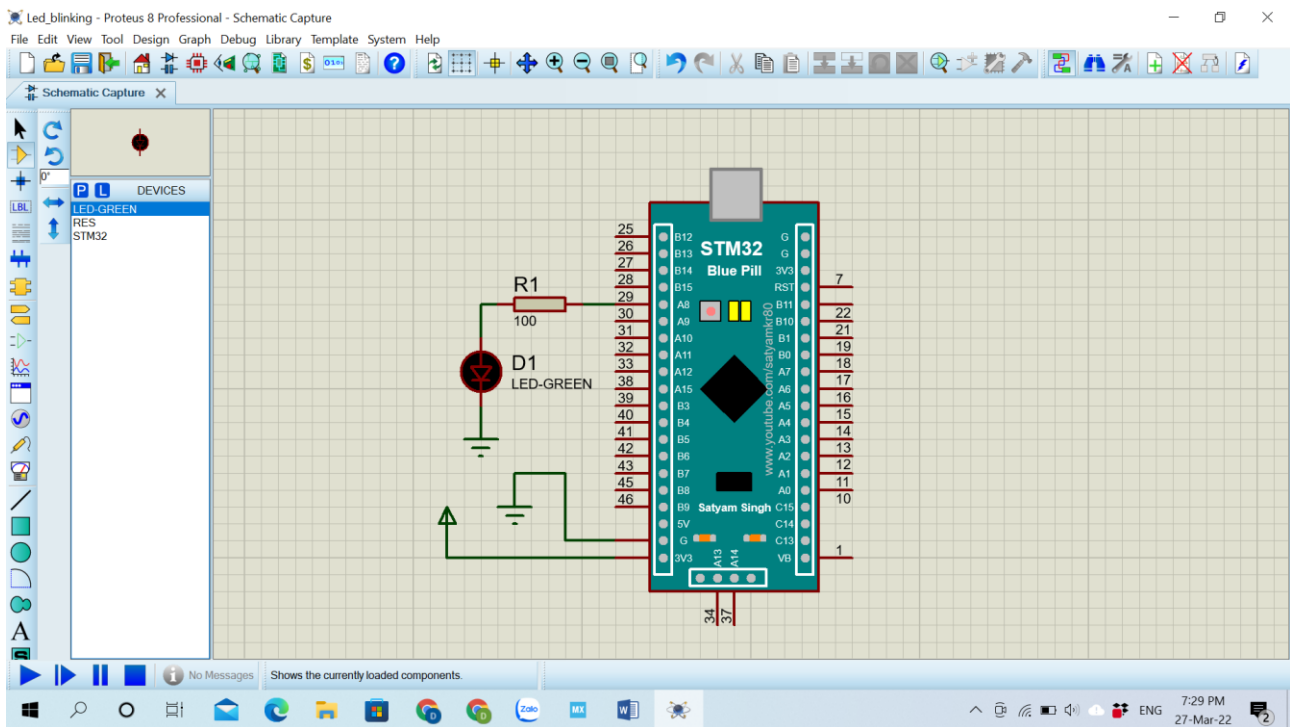
8. STM32CubeMX

STM32CubeMX tool is part of STMicroelectronics STMCube .This software tool makes the development easy by reducing development effort, time and cost. STM32Cube includes STM32CubeMX which is a graphical software configuration tool that allows the generation of C initialization code using graphical wizards. That code can be used in various development environments like keil uVision, GCC, IAR etc. You can download this tool from the [following link](#).

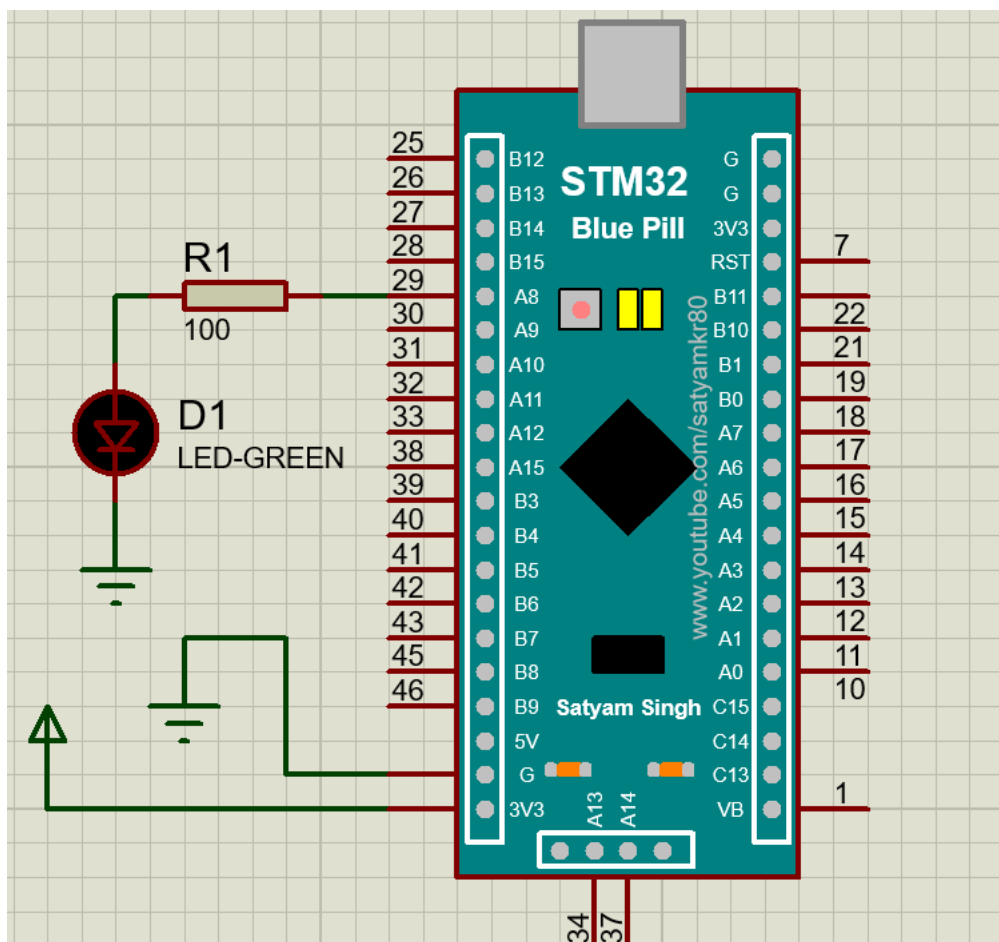
STM32CubeMX has following features

- Pin out-conflict solver
- A clock-tree setting helper
- A power-consumption calculator
- An utility performing MCU peripheral configuration like GPIO pins, USART etc
- An utility performing MCU peripheral configuration for middleware stacks like USB, TCP/IP etc

9. Steps



9.1. Circuit



Notes: Chú ý: điều chỉnh nguồn cấp cho Blue pill là 3.3V (chọn nguồn → Design →config power rails →Vcc nhập giá trị 3.3V →OK

10. How to Program STM32F103C8T6 using Keil uVision?

In this tutorial, I will show you How to Program STM32F103C8T6 using Keil uVision and STM32CubeMX Code Generator Software. You will learn how to install the Software (both Keil and STM32CubeMX), configure the software as per our requirements and upload the code using ST-Link Debugger and Programmer.

Coming to the hardware, we have to use a separate hardware for programming (as well as debugging) the STM32 Blue Pill Board. This hardware is called as ST-LINK/V2. It is an in-circuit programmer and debugger developed by STMicroelectronics. Although the original programmer will be expensive, there are many third-party programmers that are compatible with the original programmer and cost very less.

There is another important software called the STM32CubeMX. This is also provided by STMicroelectronics and is used to generate the code for a specific STM32 MCU as well as a particular IDE.

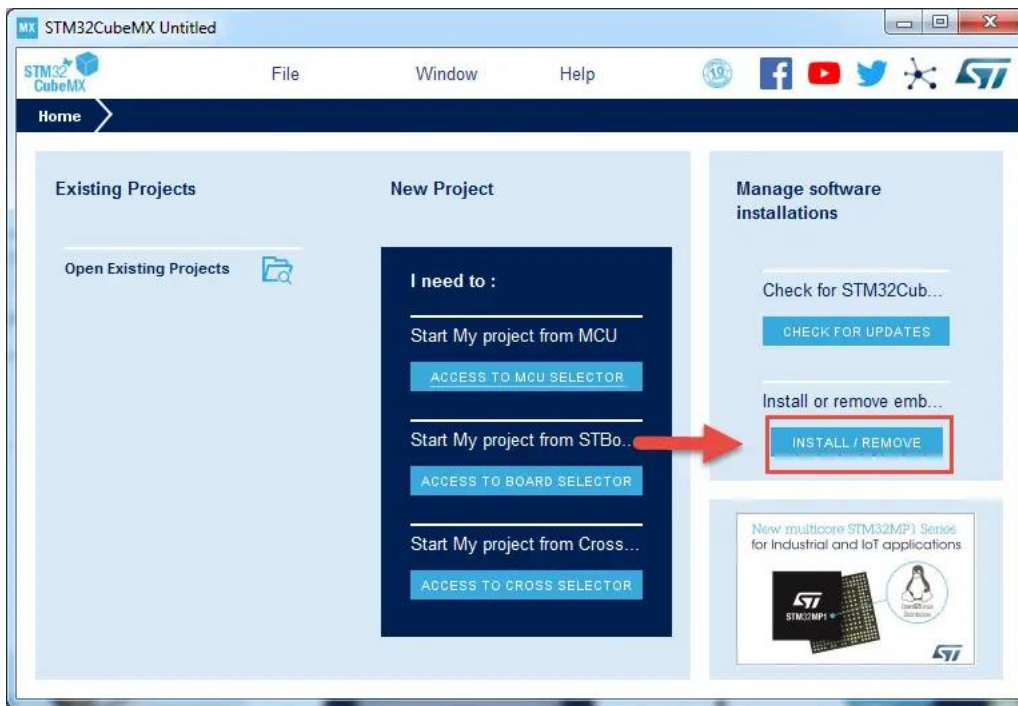
I will show you how to use all these software and hardware so that you can easily program STM32F103C8T6 using Keil uVision and STM32CubeMX.

10.1. Download and Install STM32CubeMX

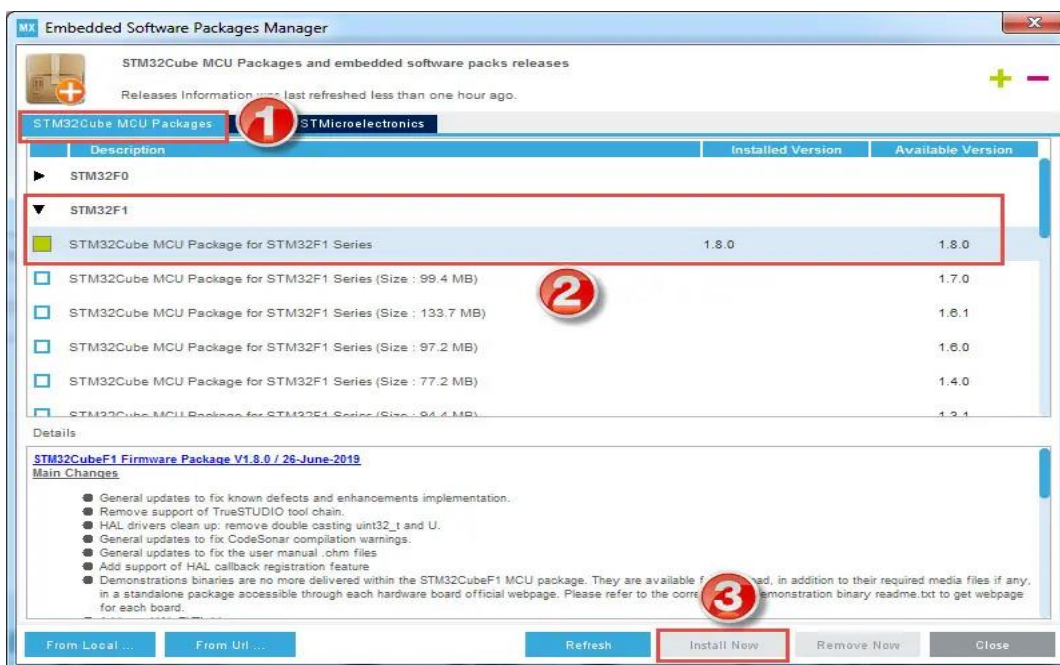
First, let us start with the STM32CubeMX Code Generator Tool. Download this software from [this](#) official STMicroelectronics website. You have to login to ST in order to download the software. At the time of writing this tutorial, the version of STM32CubeMX is 5.6.1. Install the software and open it.



On the right side in the application, click on the install or remove embedded software packages option. A new window will open.

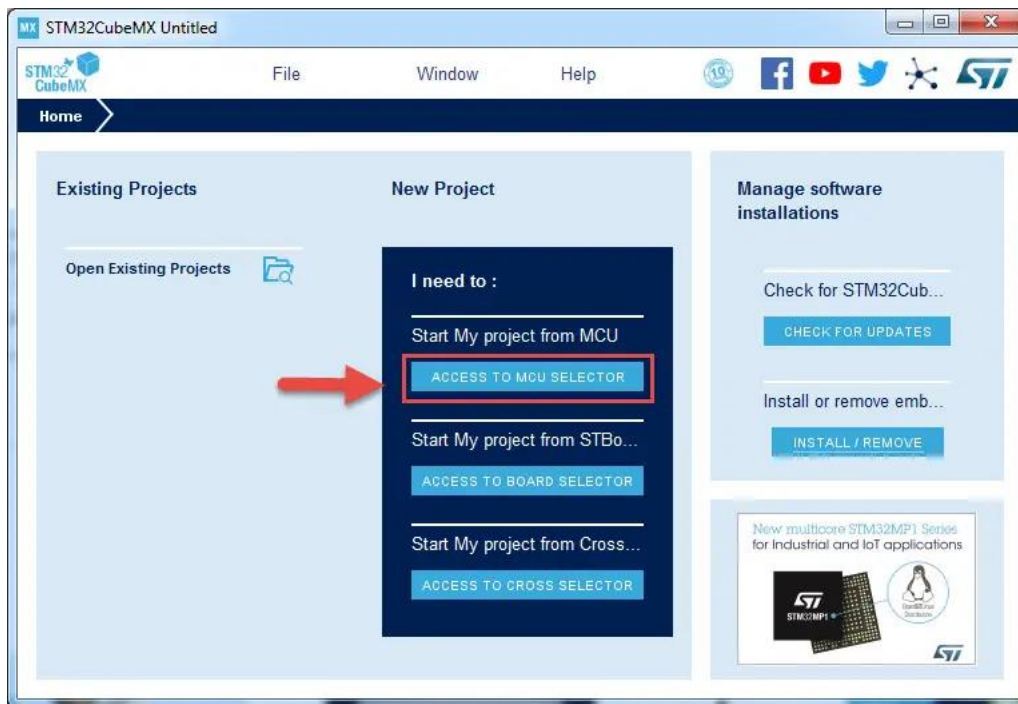


In that, select the STM32Cube MCU Packages option and expand the STM32F1 tab (since our MCU is STM32F103C8T6). The latest package at the time is 1.8.0. Select the latest version and click on Install Now option at the bottom. It will download and install the MCU Package. After completion, you can close this window.

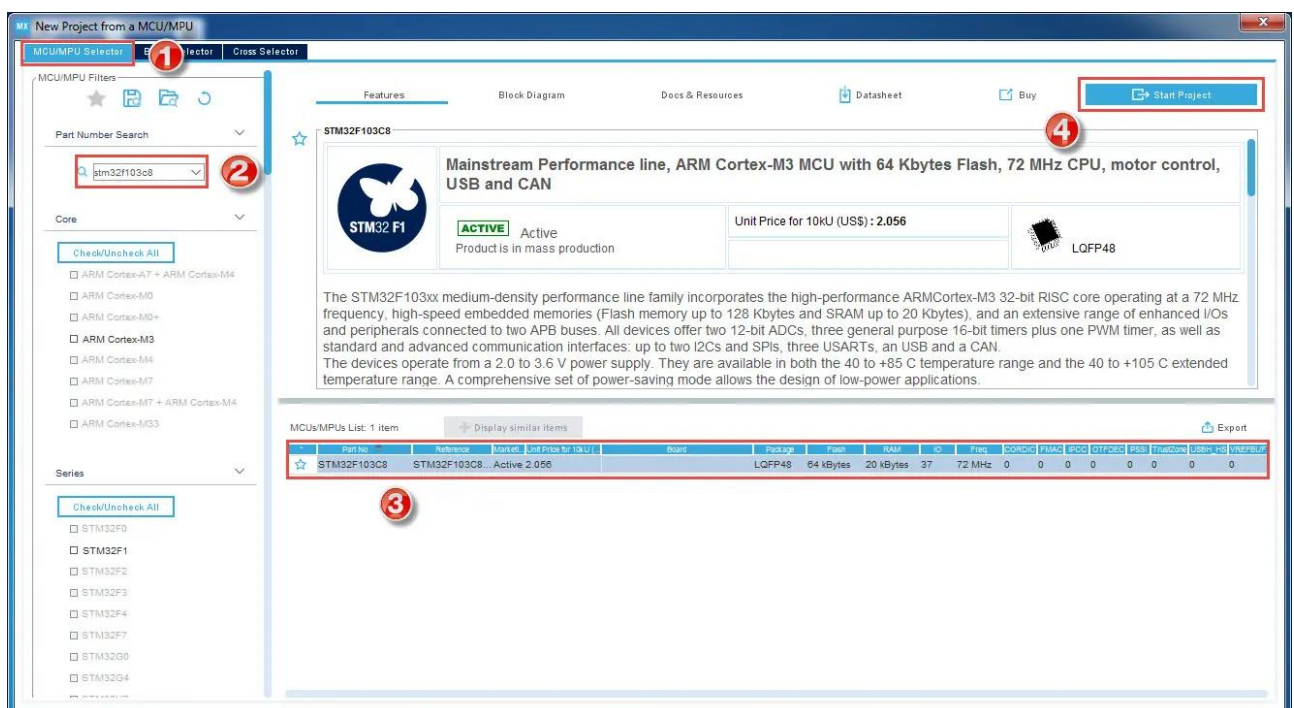


11. New STM32CubeMX Project

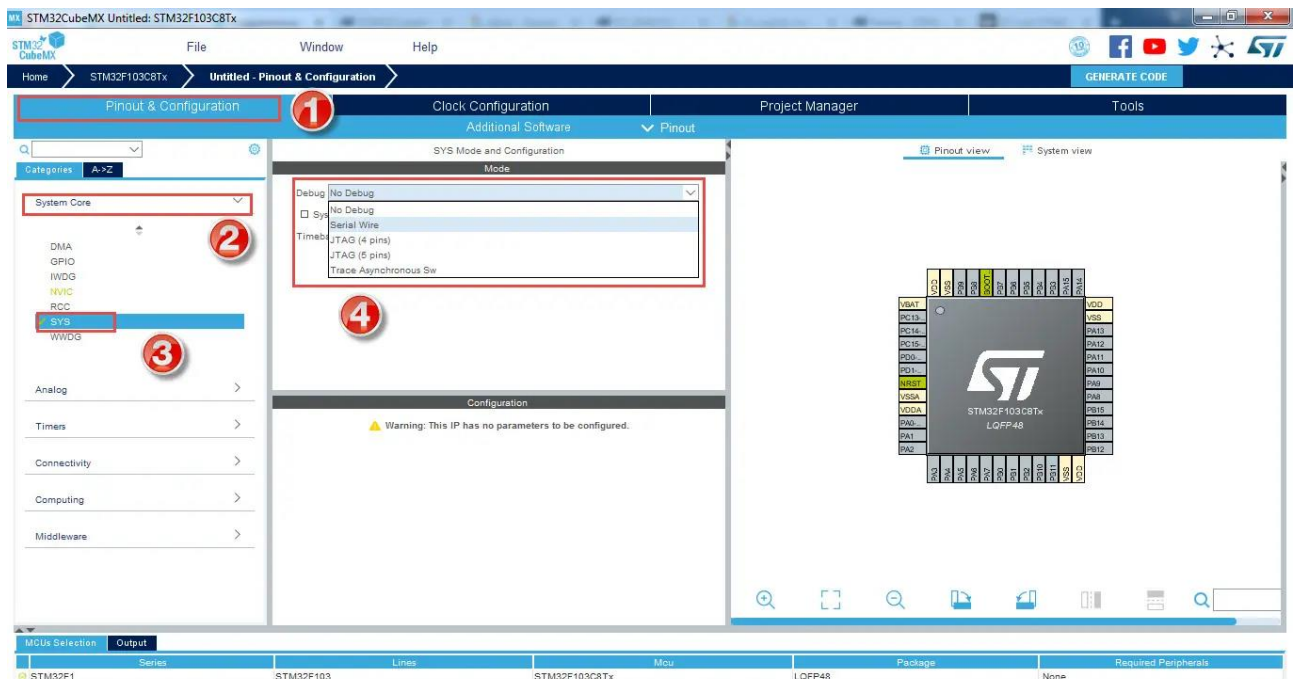
Now, in the main window, select the ACCESS TO MCU SELECTOR option. A new window will open for us to select the MCU that we are working



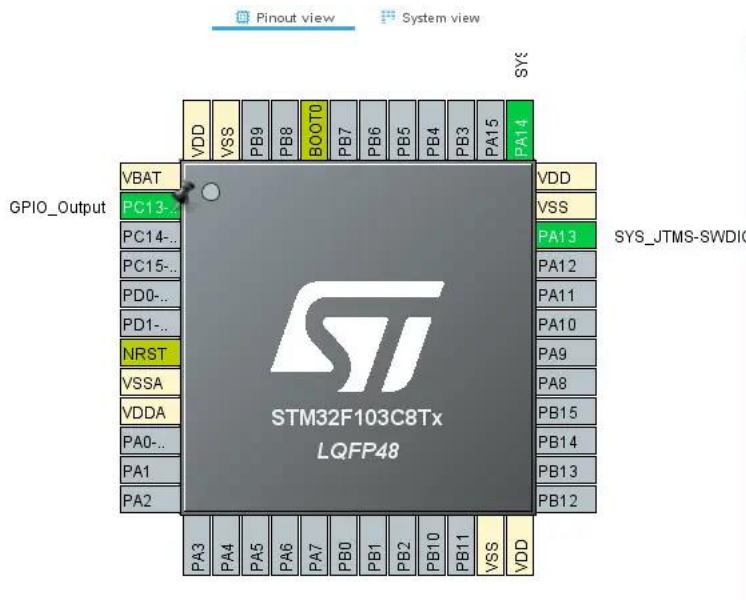
In the new window, under the MCU/MPU Selector tab, search for “STM32F103C8”. On the right side, our MCU will be listed. Select that MCU and on the top right corner click on the Start Project option.



The main project window will open with a pinout of STM32F103C8T6 MCU on the right and configuration options on the left. In the Pinout and Configuration Tab, expand the System Core block and select the SYS option. Select the Debug as Serial Wire. As soon as you select this option, you can observe that the Pins PA13 and PA14 are SWDIO and CK i.e. Serial Wire Debug Data and Clock pins.



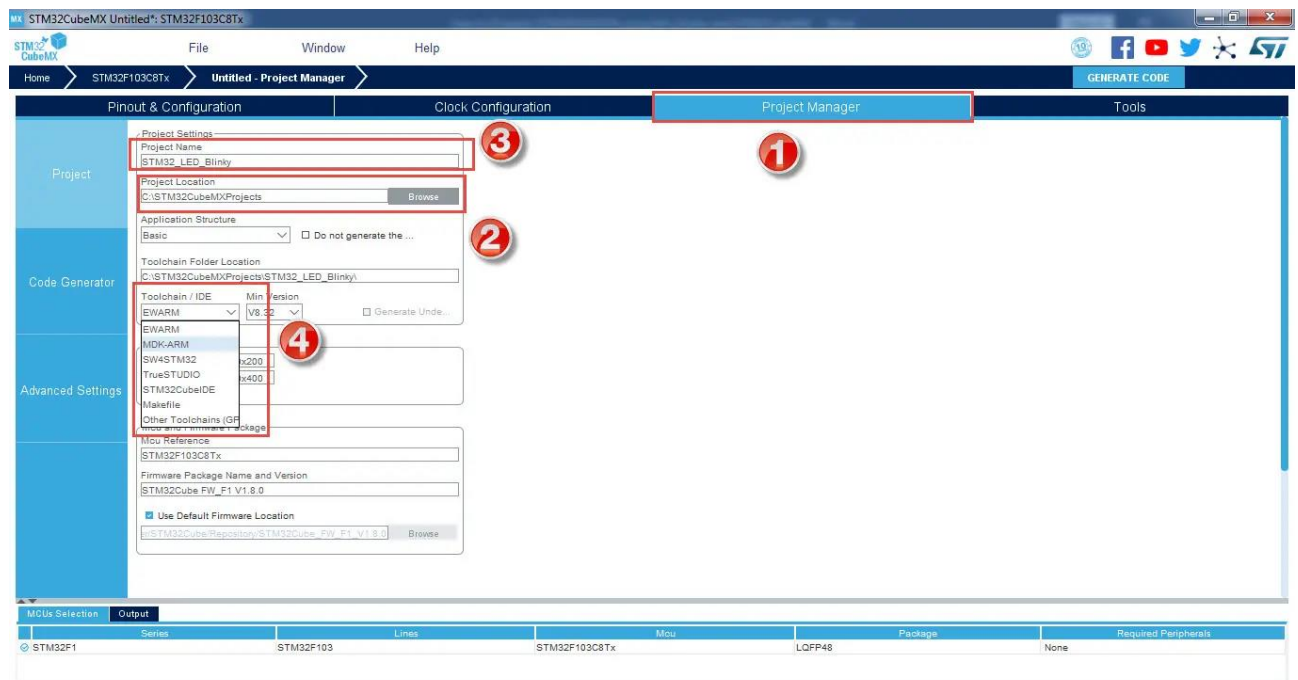
The next step is to configure the LED present on the Blue Pill Board. We know from the schematic of the STM32 Blue Pill that a Green LED is connected to PC13. Hence, zoom into the Pinout view of the STM32F103C8T6 MCU and on the PC13 Pin, make a left click and select GPIO_Output option. We can change the label of this pin by right clicking on the PC13 Pin and selecting Enter User Label option. I have used the label “LED_Green”.



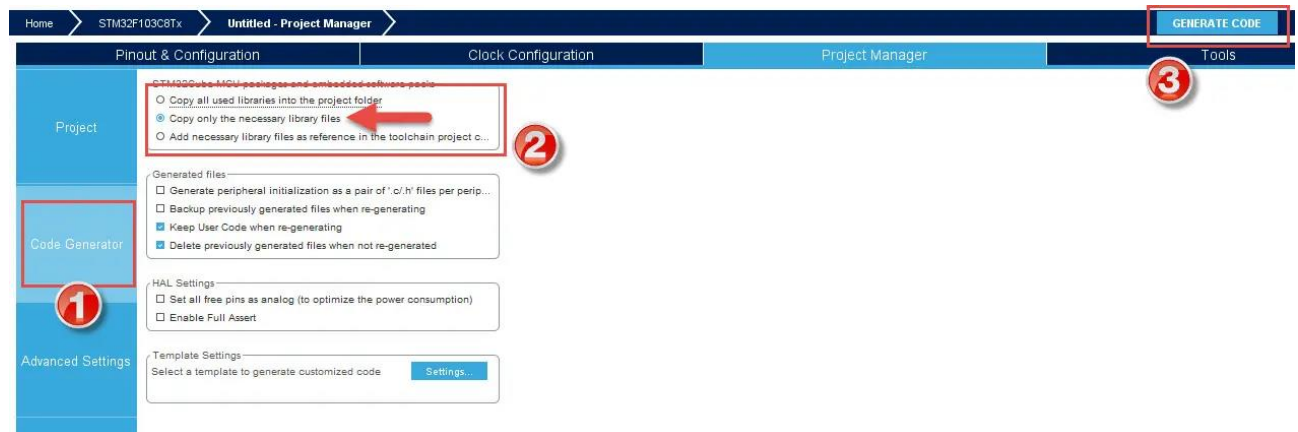
This completes the basic setup for the project. Other thing that we can configure is the Clock but the STM32 MCU uses the internal RC Oscillator by default (configured in the SystemInit Function). So, we can leave the Clock Configuration as it is.

In the next step, select the Project Manager Tab and select an appropriate location and name for your project. In my case, I have named the project as “STM32_LED_Blinky”.

Also, select the Toolchain / IDE as MDK-ARM, which is nothing but the Keil uVision IDE.



After selecting the toolchain, click on the Code Generator option and select the copy only the necessary library files option. After this, you can click on GENERATE CODE option. The tool will create a project based on the IDE selected and also copies the necessary library files for the project. After the code is generated, you can close the STM32CubeMX Software.



12. A Brief Note on ST-LINK Debugger and Programmer

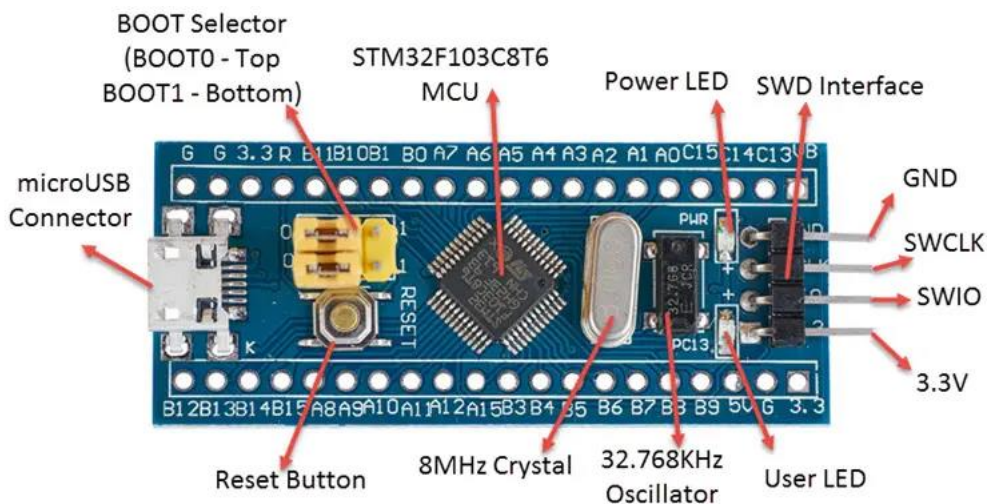
Before proceeding further, let us take a brief look at the ST-LINK Programmer and Debugger. Its latest revision is known as ST-LINK/V2. The cheaper version of the ST-LINK/V2 Programmer is shown in the image below.



One end of the programmer is a USB Male Connector, which you can plug-in to a computer. The other end of the programmer consists of the JTAG / Serial Wire Debugging (SWD) Interface. The pinout of this interface is neatly printed on top of the debugger Module.

RST	1	2	SWCLK
SWIM	3	4	SWDIO
GND	5	6	GND
3.3V	7	8	3.3V
5.0V	9	10	5.0V

From these pins, we have to use the 3.3V, GND, SWDIO and SWCLK pins. If you remember the pinout of the STM32F103C8T6 Blue Pill Board, the SWD interface is neatly placed on the right side.

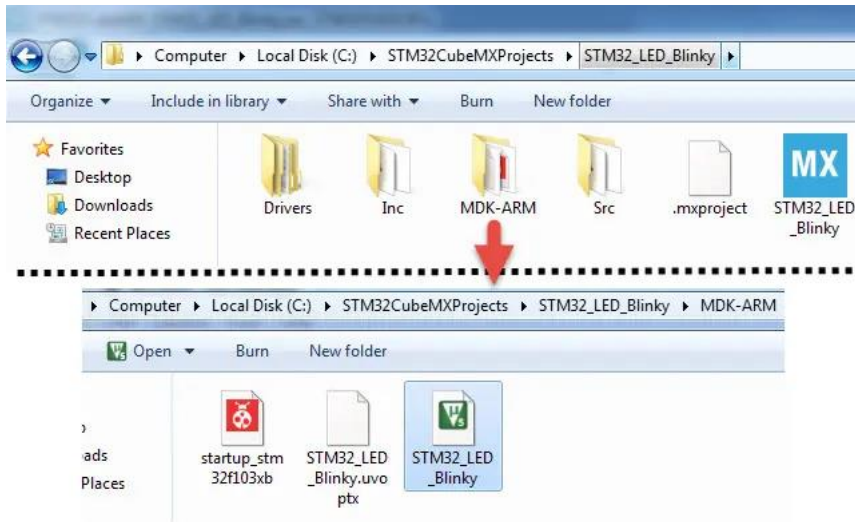


13. Keil Project

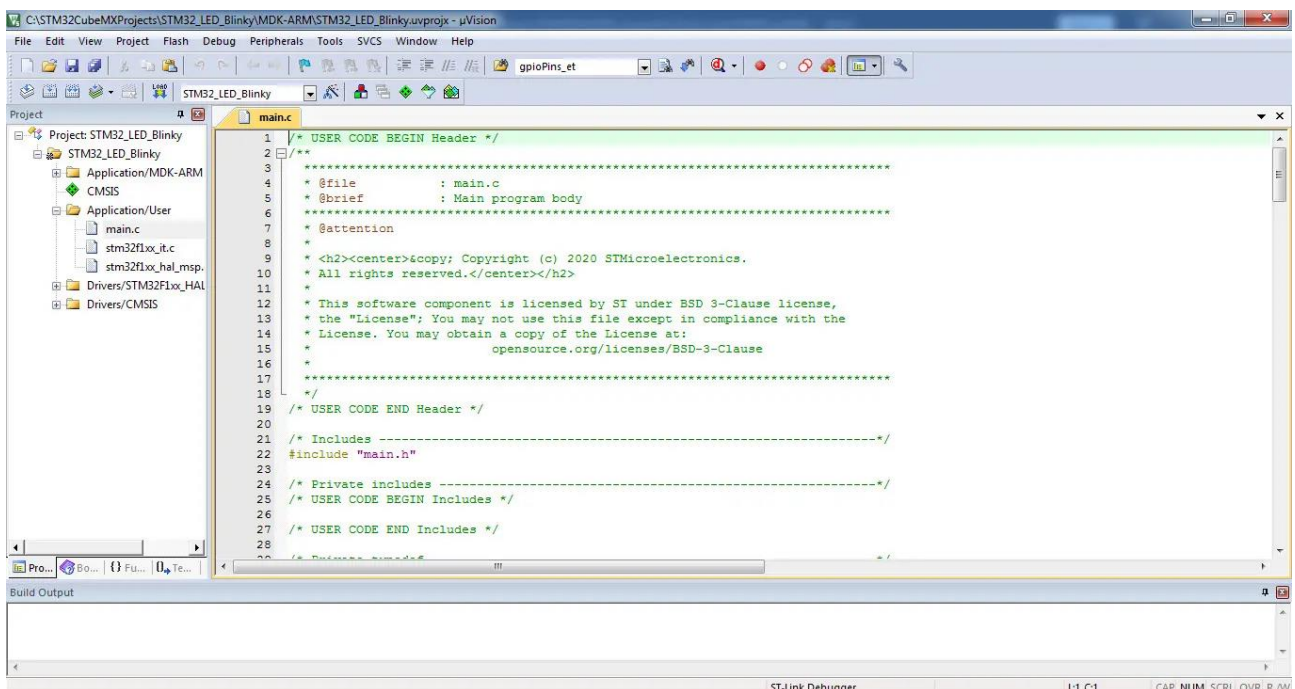
After installing, the MDK, you don't need to open the Keil uVision as we will be doing that from the previously generated code using STM32CubeMX.

New Keil uVision Project

Instead of creating a new Keil uVision Project, we can use the project created during the STM32CubeMX Code Generation. Open the folder where the STM32CubeMX Project was saved earlier and open the MDK-ARM Folder.



In this folder, there is a Keil MDK Project file STM32_LED_Blinky. Double click on this file and the Keil uVision will open with all the source code and library files. After this, you can open the main.c file.



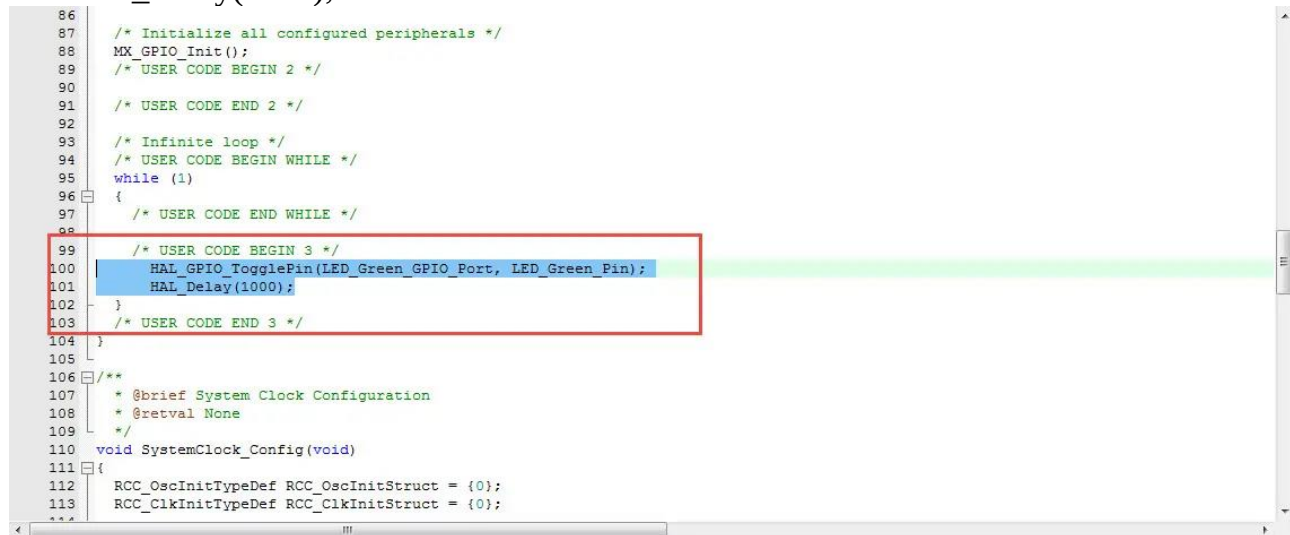
Modifying the Code for Blinking LED

The code generated using STM32CubeMX is just the initialization code and doesn't do anything other than initializing some peripherals as per the user configuration. In order to do

implement an application, like Blinking an LED for example, you have to add the necessary code.

In the main.c file, scroll down to the main function and in the while (1) loop, add the following code.

```
HAL_GPIO_TogglePin(LED_Green_GPIO_Port, LED_Green_Pin);  
HAL_Delay(1000);
```



```
86  
87 /* Initialize all configured peripherals */  
88 MX_GPIO_Init();  
89 /* USER CODE BEGIN 2 */  
90  
91 /* USER CODE END 2 */  
92  
93 /* Infinite loop */  
94 /* USER CODE BEGIN WHILE */  
95 while (1)  
96 {  
97     /* USER CODE END WHILE */  
98  
99     /* USER CODE BEGIN 3 */  
100     HAL_GPIO_TogglePin(LED_Green_GPIO_Port, LED_Green_Pin);  
101     HAL_Delay(1000);  
102 }  
103 /* USER CODE END 3 */  
104  
105  
106 /**  
107  * @brief System Clock Configuration  
108  * @retval None  
109  */  
110 void SystemClock_Config(void)  
111 {  
112     RCC_OscInitTypeDef RCC_OscInitStruct = {0};  
113     RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};  
114 }
```

This code will Blink the LED with 1 second delay. Now, you can “Build” the project using either F7 or the shortcut in the IDE. Keil uVision will now compile the code and display the result of the compilation at the bottom. If there are any errors or warnings, the line number will be displayed at which the error or warning is present so that you can easily correct the mistakes.

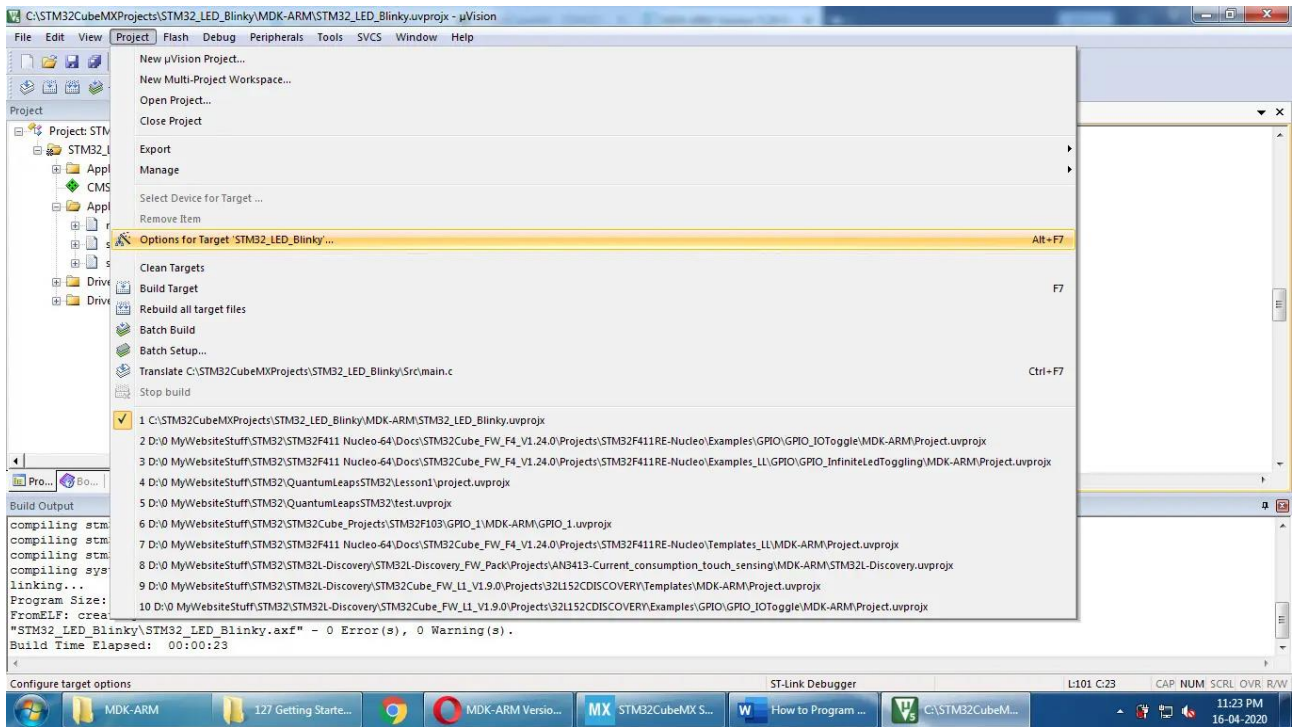
14. Connect ST-LINK with STM32F103C8T6

Connect the SWDIO pin of ST-LINK/V2 to the SWIO pin of the STM32 Blue Pill. Also connect the SWCLK pin of ST-LINK/V2 to the corresponding SWCLK pin of the STM32 Blue Pill. Finally, connect the 3.3V and GND pins.

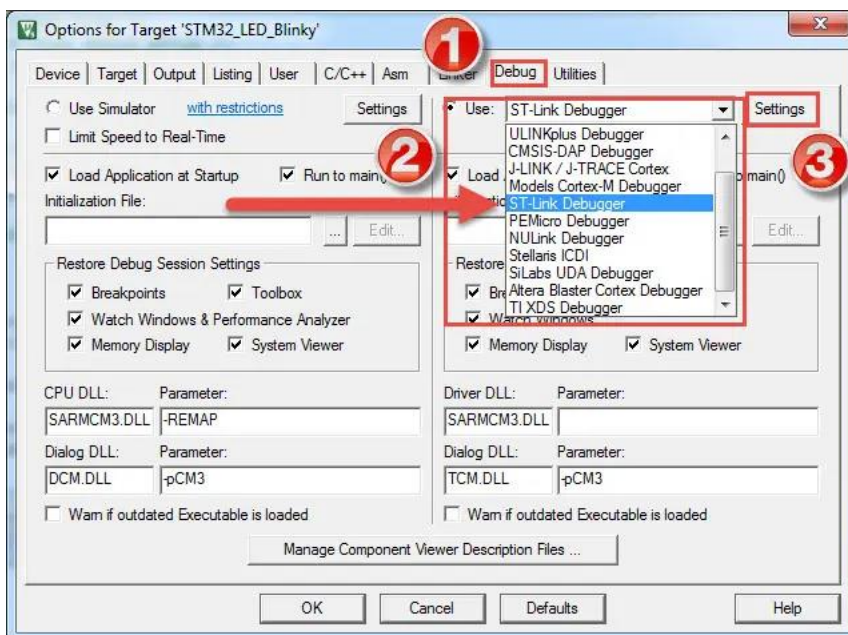
<i>ST-LINK/V2</i>	<i>STM32 Blue Pill</i>
GND	GND
3.3V	3.3V
SWDIO	SWIO
SWCLK	SWCLK

15. How to Program STM32F103C8T6 using Keil uVision?

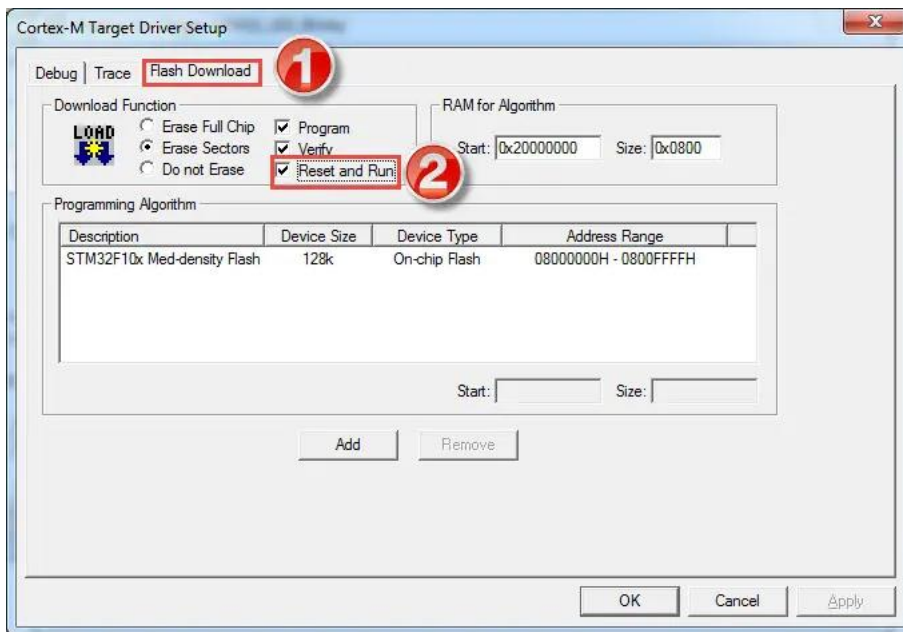
After making the above connections, we can proceed to Program STM32F103C8T6 using Keil uVision. Connect the ST-LINK/V2 Debugger to the computer. In the Keil uVision, click on Project Tab and select Options for Target..... link.



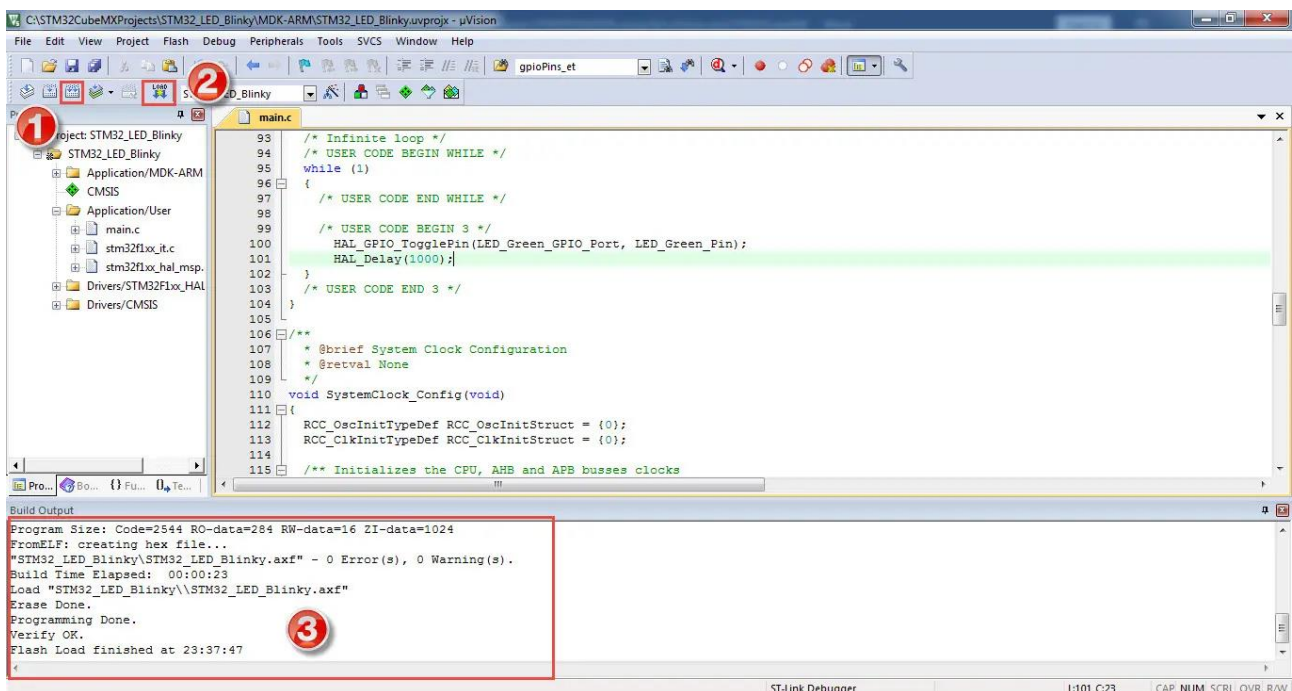
Select the Debug tab and select ST-Link Debugger. After that, click on Settings option adjacent to it.



Go to Flash Download option and check the Reset and Run option. After that, click on OK to save the settings and click on OK once again to close the Options for Target.



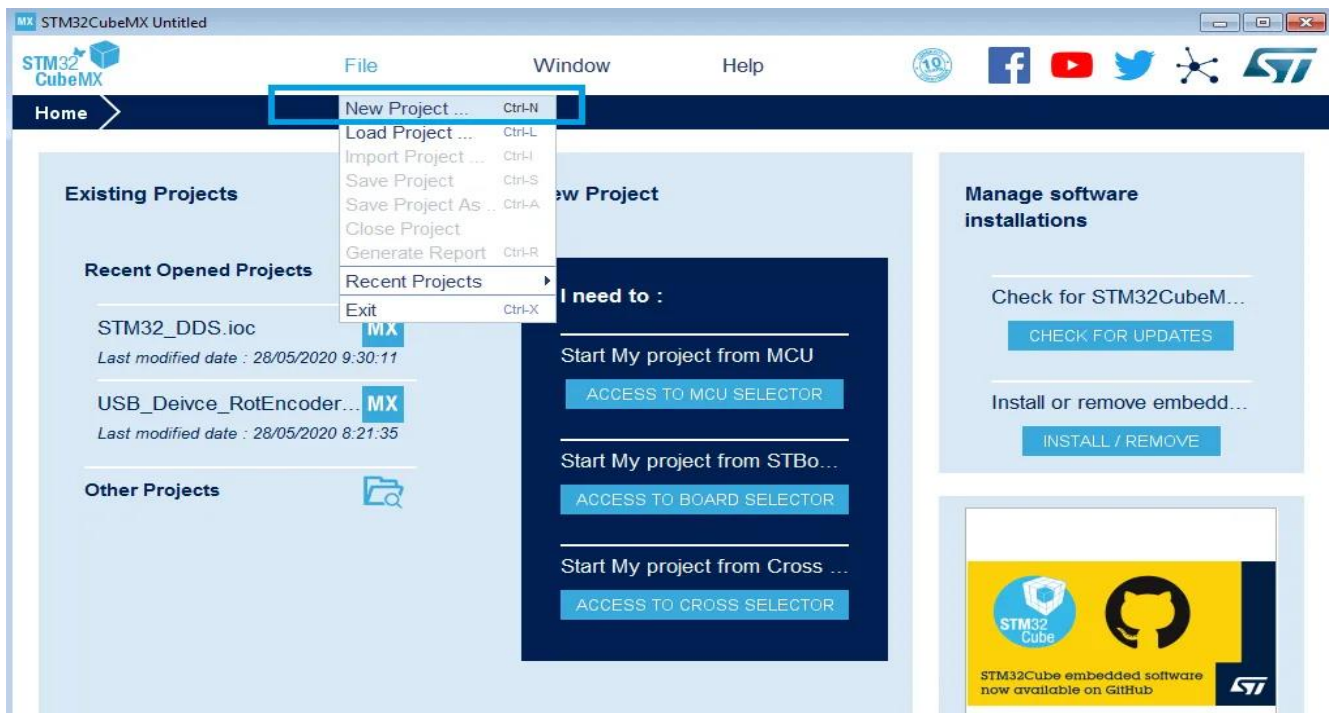
Now, if you made any changes to the code, save all the changes and select Rebuild option, which is next to the build option. If there are no errors, click on Download option. Keil uVision, with the help of ST-LINK/V2, will now program your STM32F103C8T6. You can see the status at the bottom.



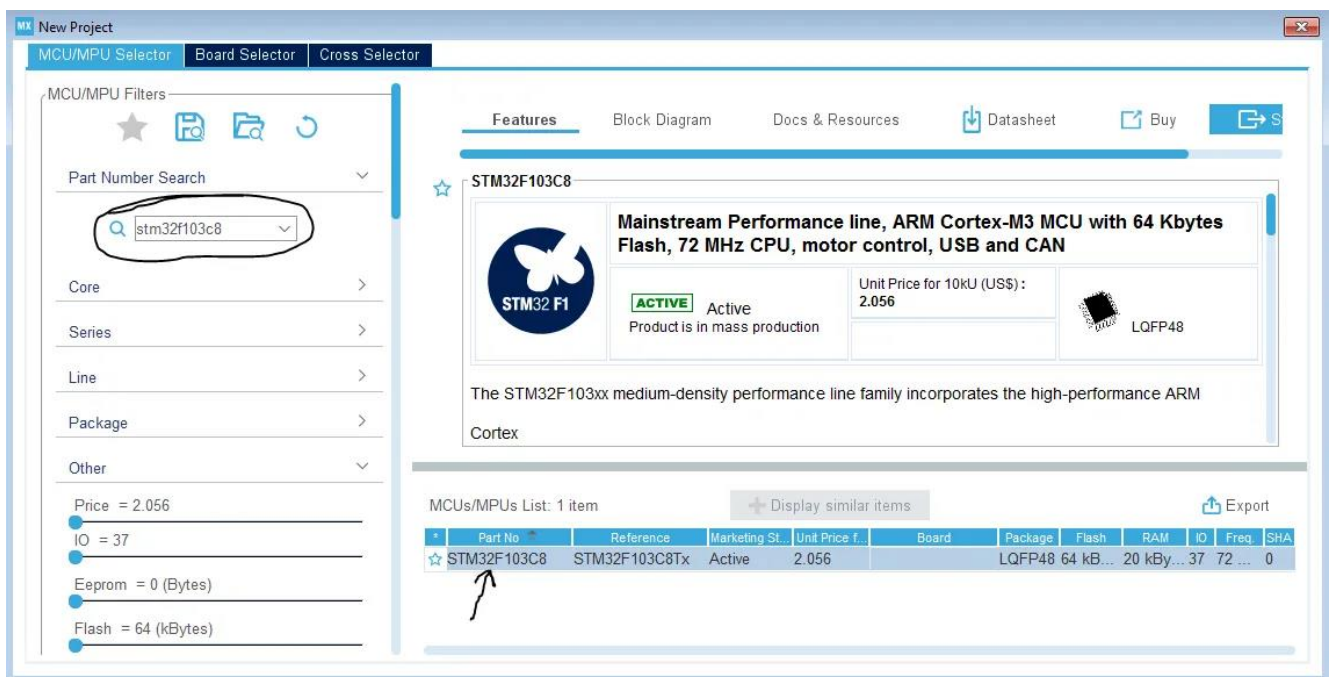
Step by step

16. STM32 CubeMX Configurations

- **Step1:** Open CubeMX & Create New Project

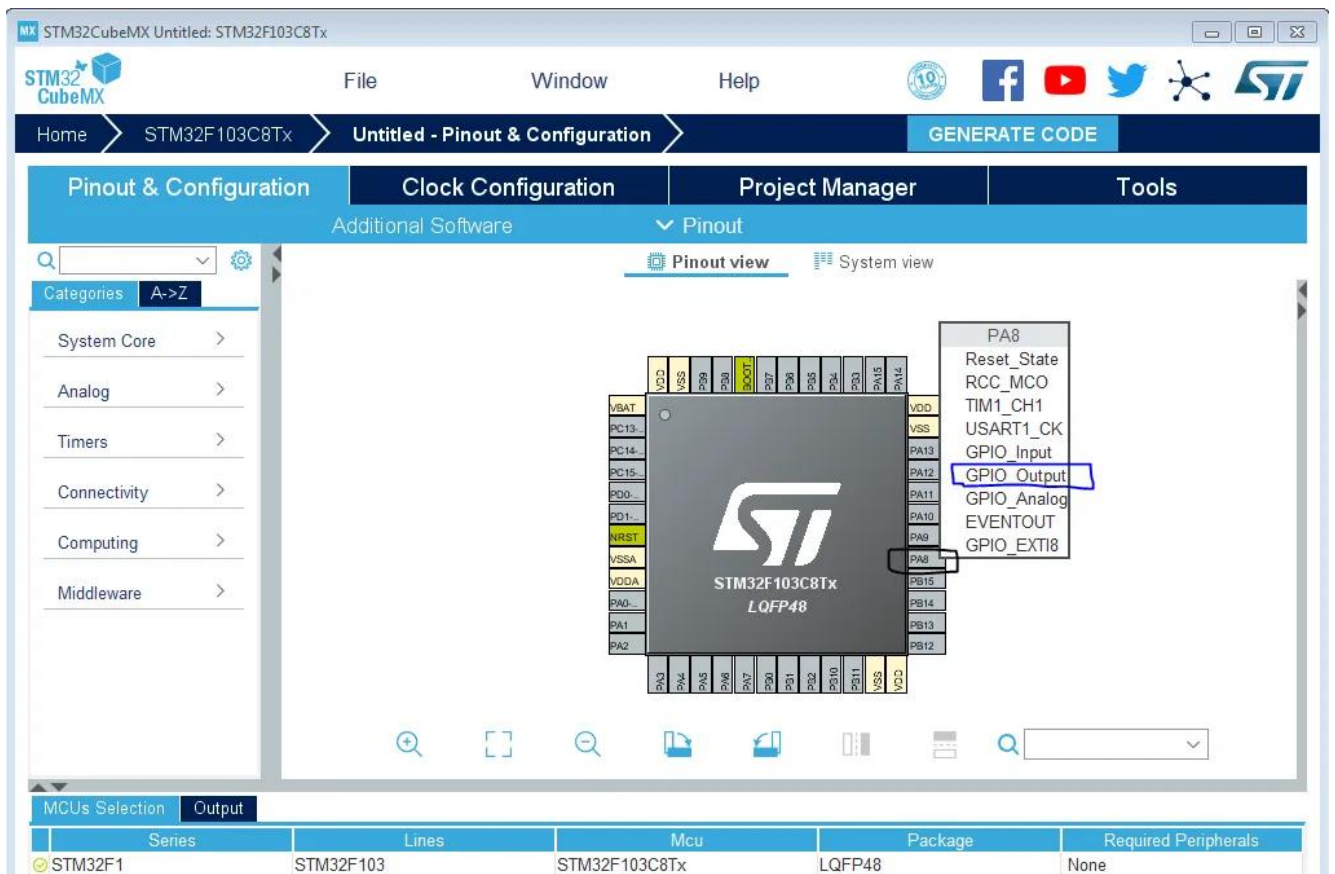


- Step2: Choose The Target MCU & Double-Click Its Name

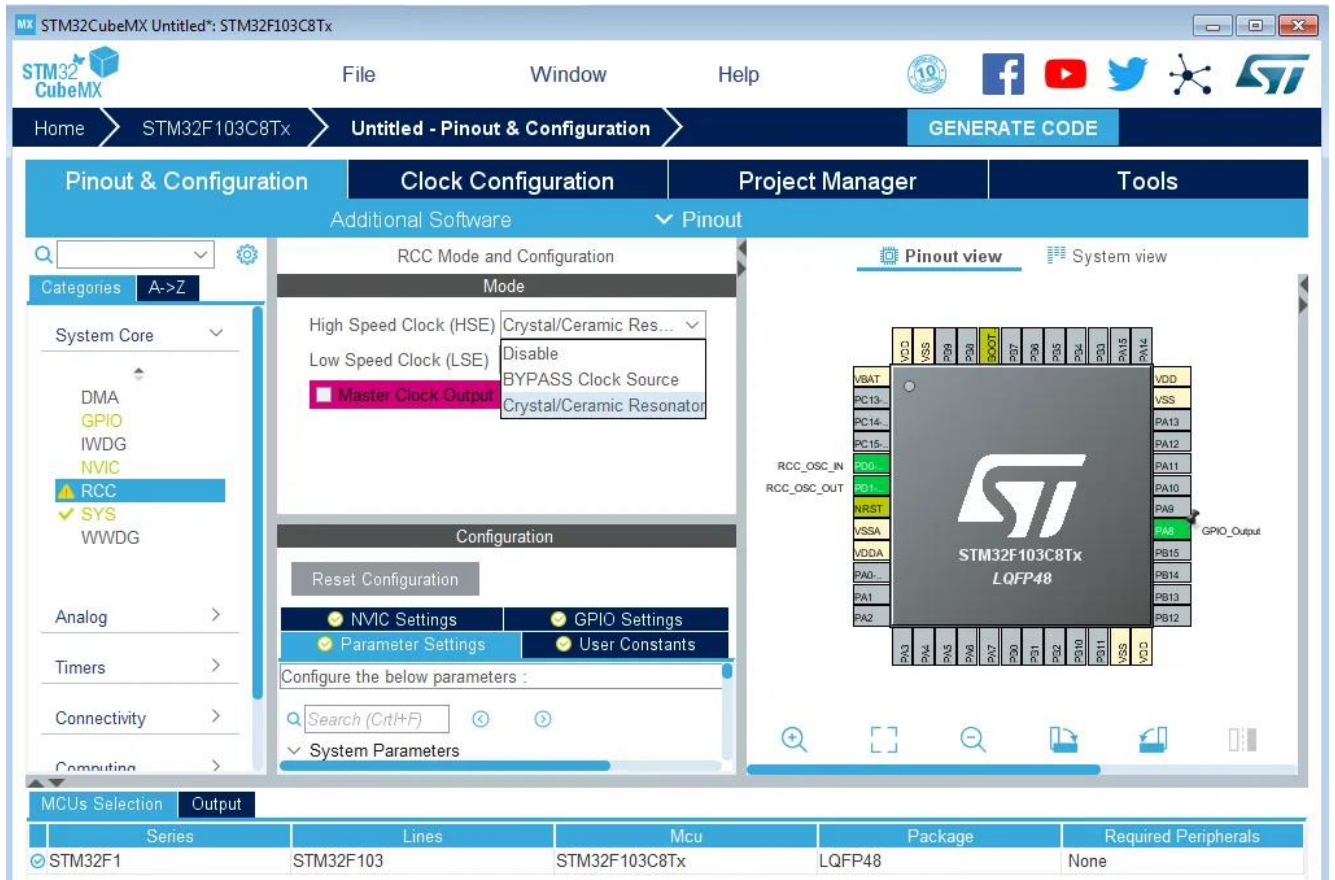


- Step3: Click On The Pin You Want To Configure As An Output & Select Output Option

Let it be **A8** pin for example!

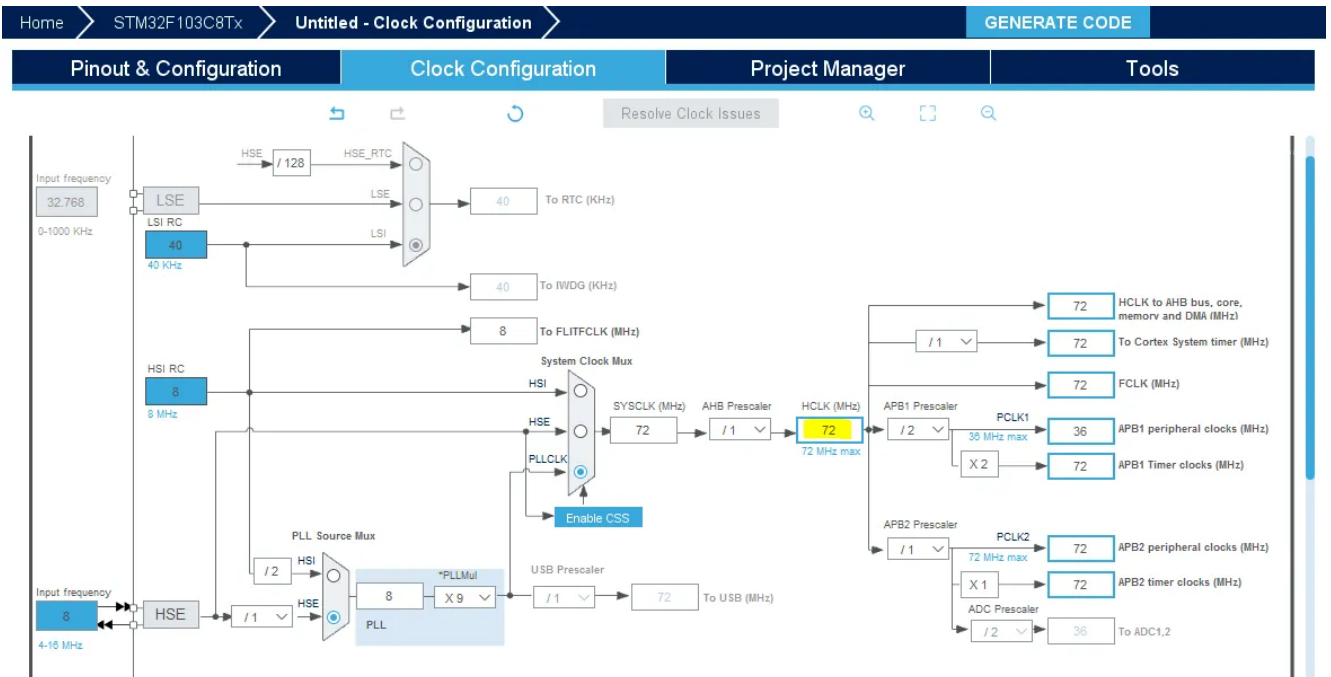


- Step4: Set The RCC External Clock Source

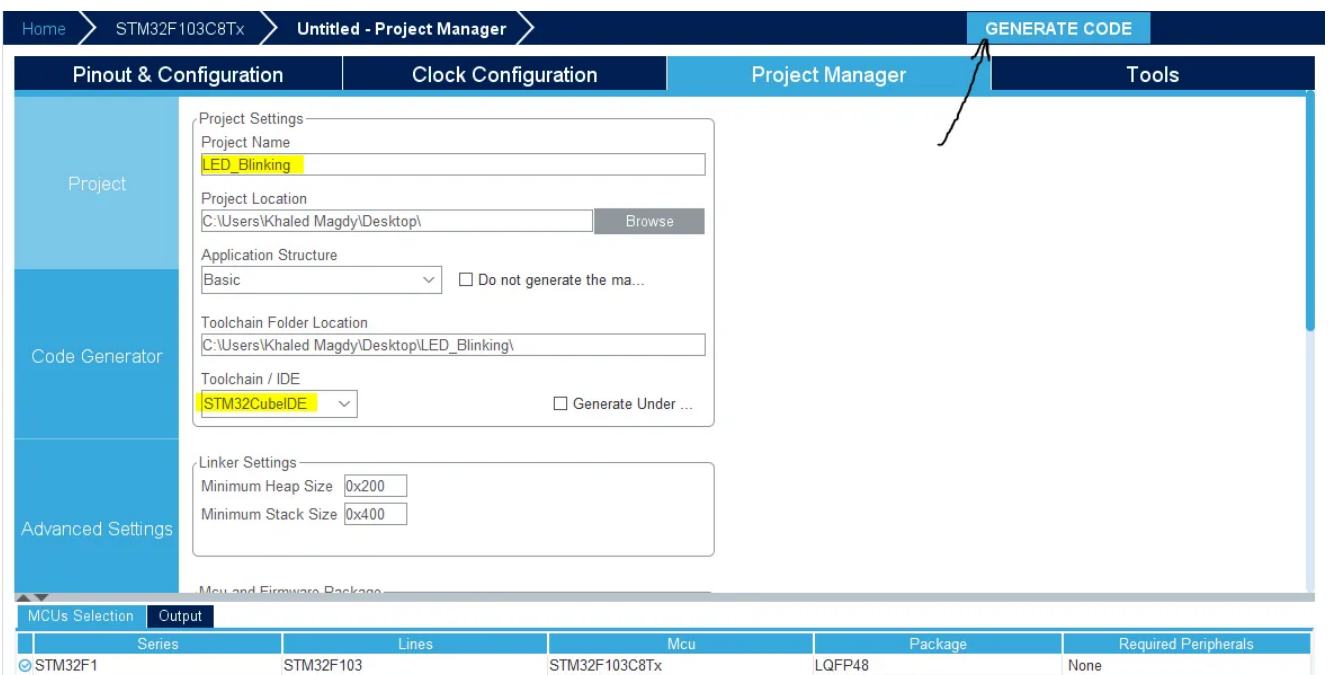


- Step5: Go To The Clock Configuration

- **Step6: Set The System Clock To Be 72MHz Or Whatever You Want**

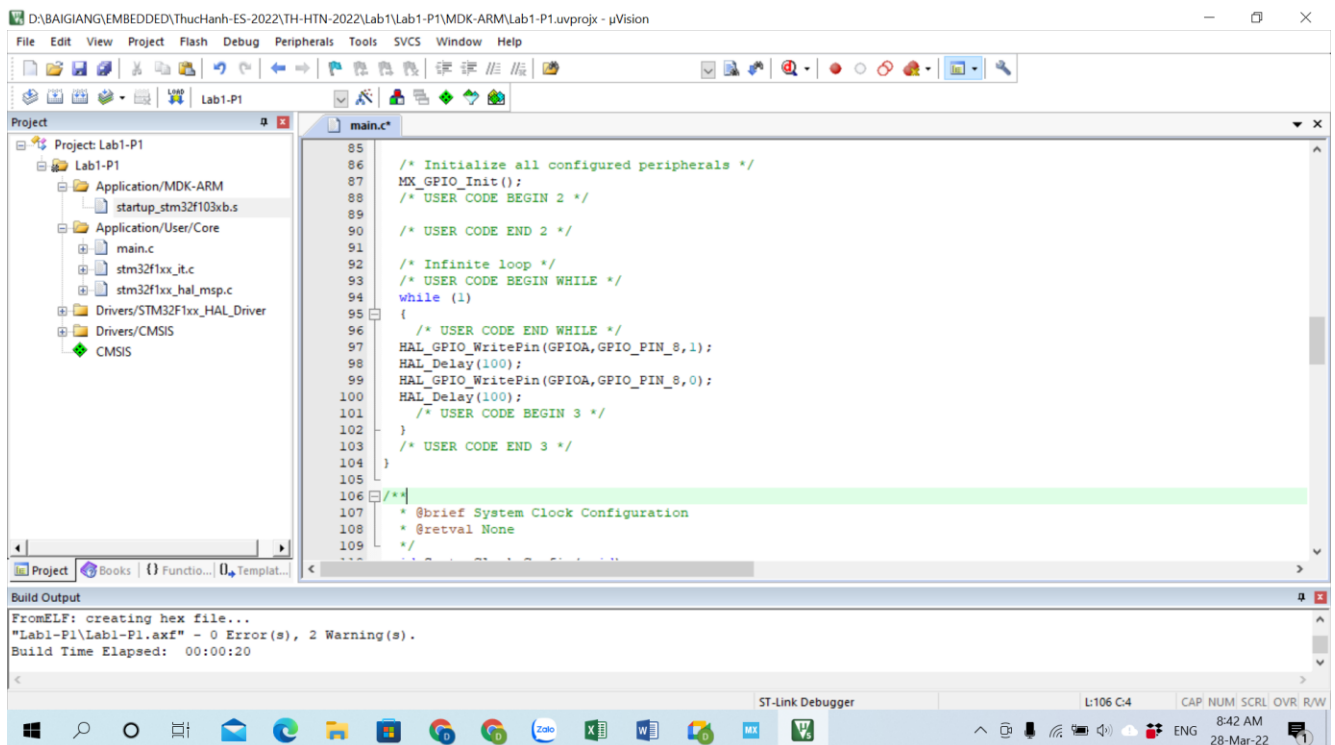


- **Step7: Name & Generate The Project Initialization Code For CubeIDE or The IDE You're Using**



Then, open the project in the IDE you're using. And head over to the `main.c` file. So we can start writing the application code and have a look at the initialization code generated by the STM32 CubeMX tool.

17. The Application Code In Keil-MDK



The main.c file in the source code directory within our projects is as shown below.

```

#include "main.h"
void SystemClock_Config(void);
static void MX_GPIO_Init(void);
int main(void)
{
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();
    while (1)
    {
    }
}

```

Notes:

- ✓ Both functions `SystemClock_Config()` and `MX_GPIO_Init()` are generated by CubeMX to configure the system clock as we've done in the GUI before and the GPIO pin which we've selected to be an output pin. The implementation of both functions is found in the file after the main function.
- ✓ We call each of them before the main loop `while(1)` as well as the `HAL_Init` function. The `HAL_Init` must be called at the beginning of your application. Its functionality is clarified in the HAL Documentation as shown below.
 - `HAL_Init()`: this function must be called at application startup to
 - initialize data/instruction cache and pre-fetch queue
 - set SysTick timer to generate an interrupt each 1ms (based on HSI clock) with the lowest priority
 - call `HAL_MspInit()` user callback function to perform system level initializations (Clock, GPIOs, DMA, interrupts). `HAL_MspInit()` is defined as "weak" empty function in the HAL drivers.

And most importantly it initializes the SysTick timer, whose ticks are used by the HAL_Delay(). The SysTick timer is set to tick @ 1000Hz or every 1mSec. So the HAL_Delay function will give you multiples of milliseconds delay.

- **HAL_Delay()**. this function implements a delay (expressed in milliseconds) using the SysTick timer.
Care must be taken when using HAL_Delay() since this function provides an accurate delay (expressed in milliseconds) based on a variable incremented in SysTick ISR.

Besides the delay function, we also need to know the HAL APIs for controlling the GPIO pins. To do basic stuff like pin read or write or port read/write, and so on.

So we'll head over again to the HAL documentation and search for the GPIO chapter, where we'll find this listing for the available APIs. The APIs are hyperlinked in the documentation file, so you can click the name of the function to go directly to its detailed description.

IO operation functions

This section contains the following APIs:

- [HAL_GPIO_ReadPin\(\)](#)
- [HAL_GPIO_WritePin\(\)](#)
- [HAL_GPIO_TogglePin\(\)](#)
- [HAL_GPIO_LockPin\(\)](#)
- [HAL_GPIO_EXTI_IRQHandler\(\)](#)

So, let's take a closer look at the GPIO_WritePin() function as we'll be using it as well.

HAL_GPIO_WritePin

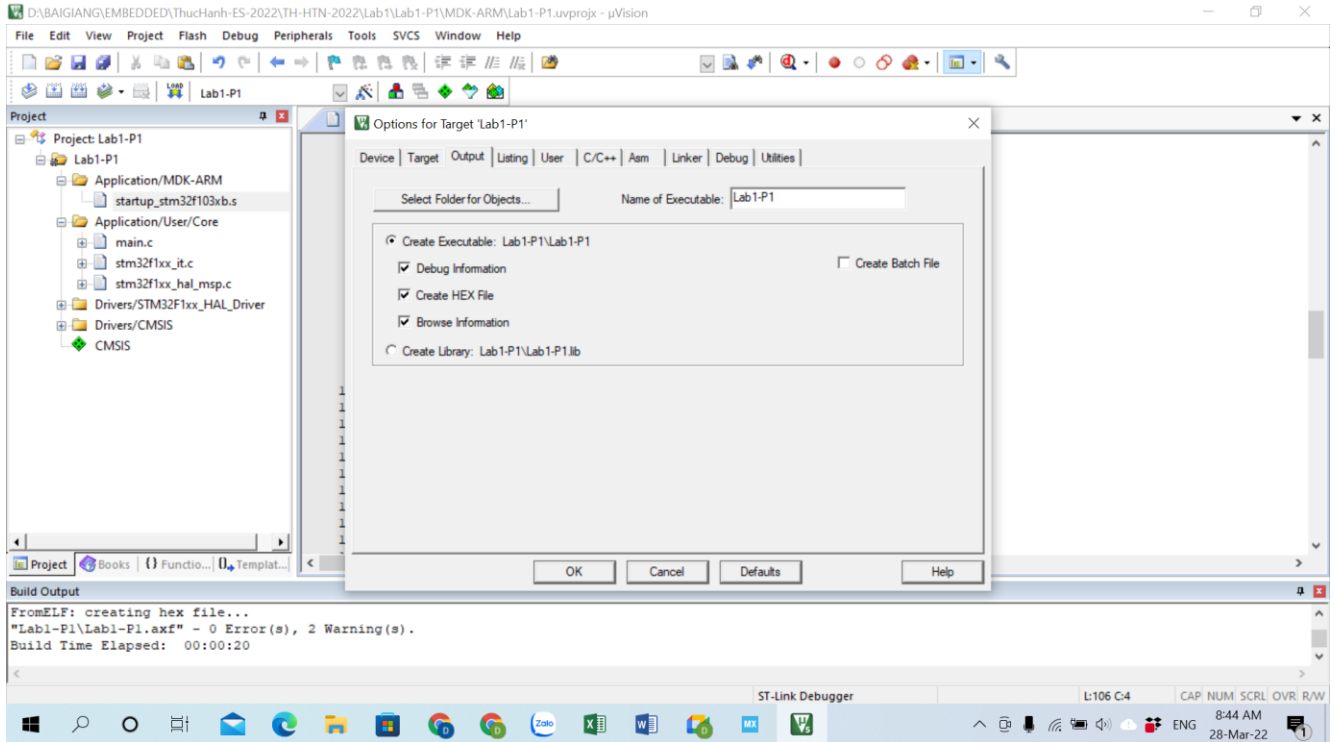
Function name	void HAL_GPIO_WritePin (GPIO_TypeDef * GPIOx, uint16_t GPIO_Pin, GPIO_PinState PinState)
Function description	Set or clear the selected data port bit.
Parameters	• GPIOx: where x can be (A..H) to select the GPIO peripheral for STM32L4 family • GPIO_Pin: specifies the port bit to be written. This parameter can be one of GPIO_PIN_x where x can be (0..15).

After reading the documentation and getting familiar with the available APIs, you are ready to go. In our blinking LED example, we won't need more than the GPIO_WritePin and HAL_Delay functions. And here is the full application code for this LAB.

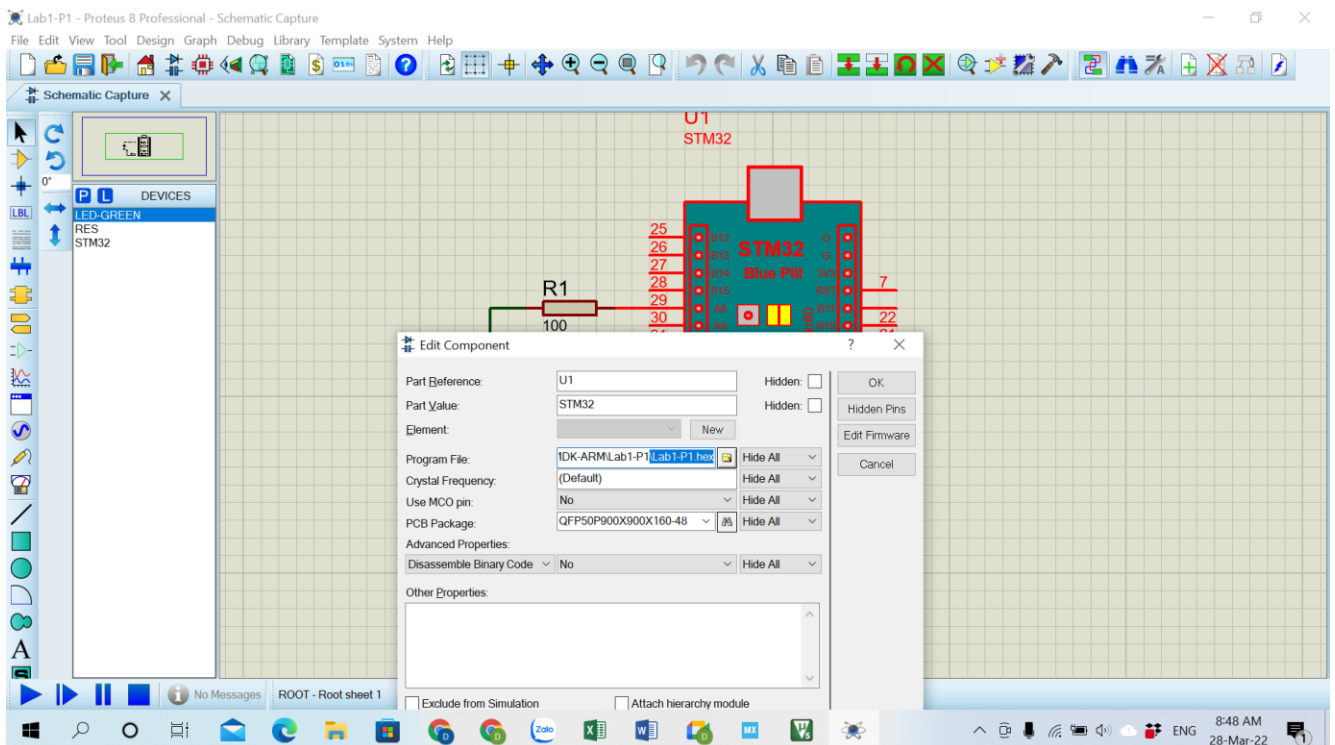
```
int main(void)
{
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();
    while (1)
    {
        // LED ON
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8, GPIO_PIN_SET);
        HAL_Delay(100);
    }
}
```

```
// LED OFF
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8, GPIO_PIN_RESET);
HAL_Delay(100);
}
}
```

- Tạo file Hex: click Options for target → check :Create Hex File → OK



- Build program: click Build (F7)
- Load Hex file into Blue Pill Board



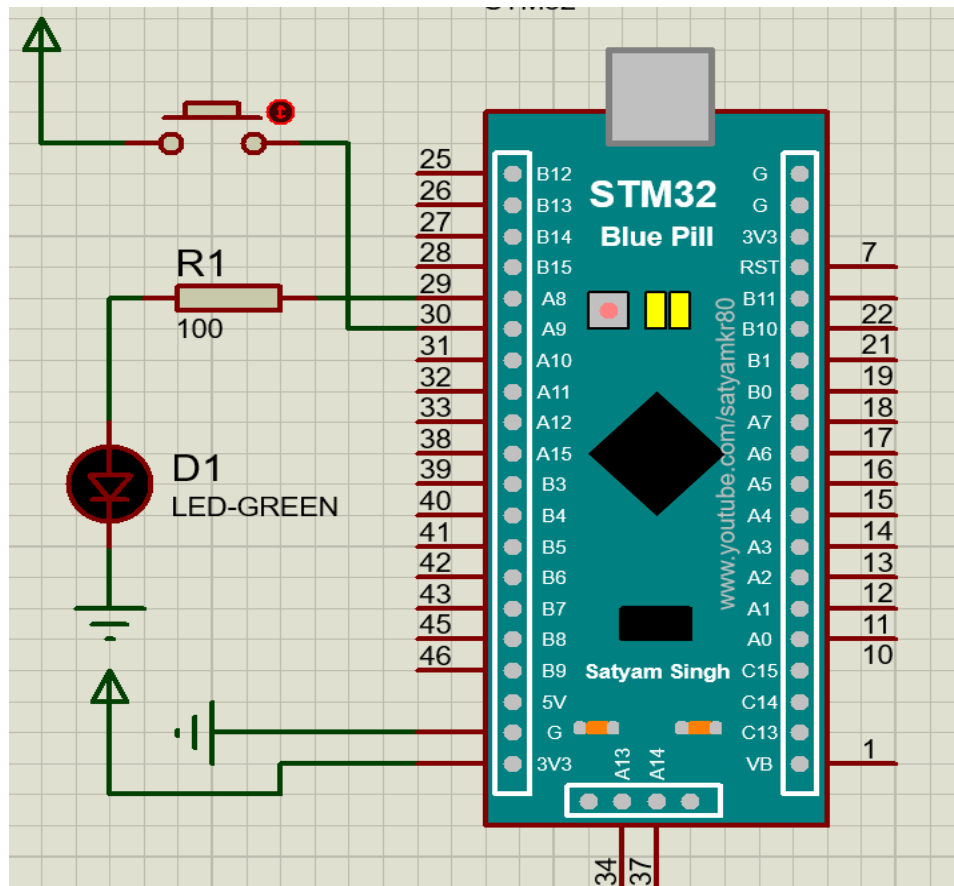
- Click Run

18. STM32 GPIO Pin Read LAB – Digital Input

In this LAB, we'll configure a GPIO pin to be output. Another one to be an input. Then, we'll write a simple application to switch an LED ON when a Switch is pressed. You'll learn all the steps to configure the STM32 CubeMX and flash the code from CubeIDE to the board and start testing.

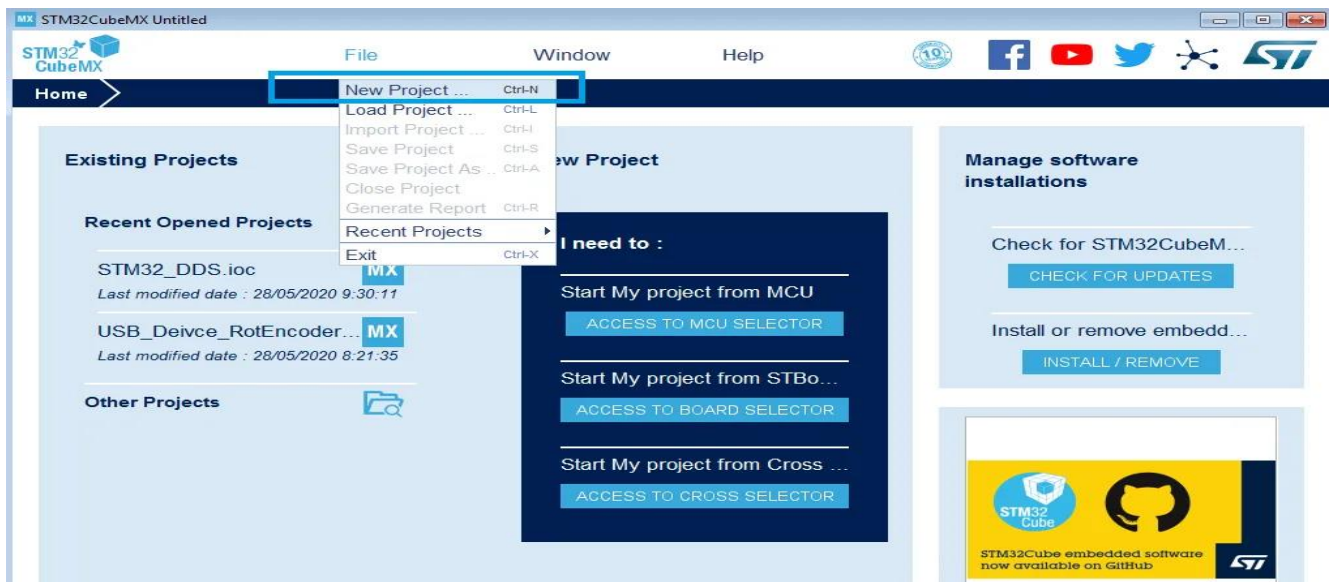
- ✓ Configure GPIO Output Pin & Input Pin Within CubeMX Tool
- ✓ Use HAL_GPIO_ReadPin To Read The Push Button State
- ✓ Use HAL_GPIO_Write To Change The Pin State

18.1. Circuit

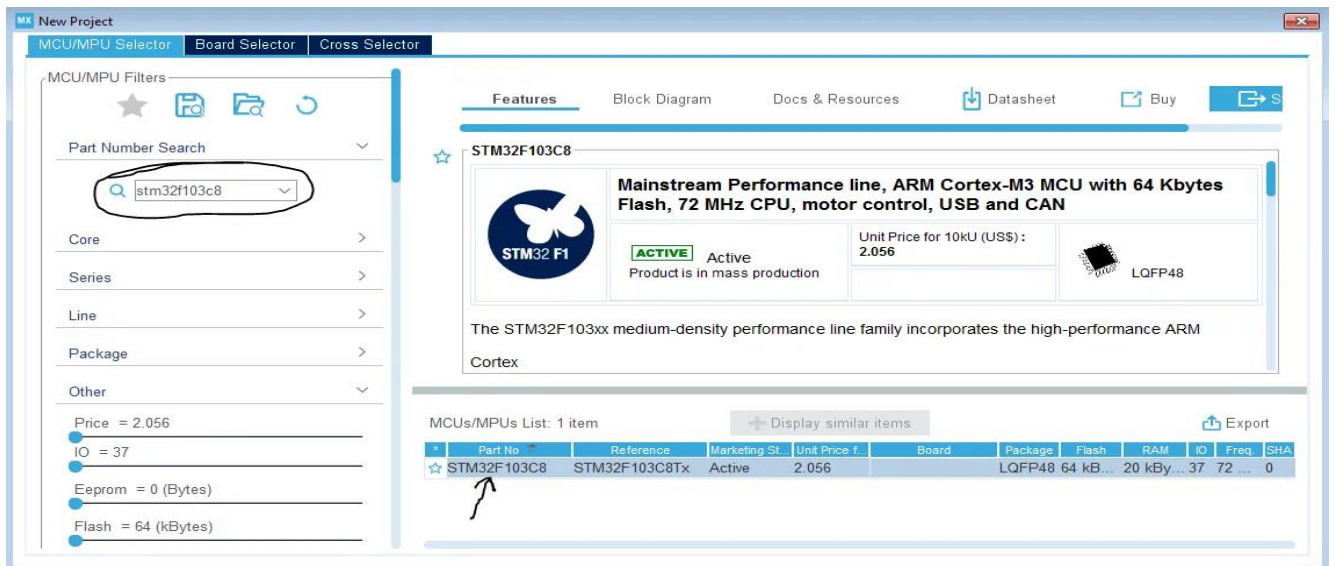


18.2. STM32 CubeMX Configurations

- **Step1:** Open CubeMX & Create New Project

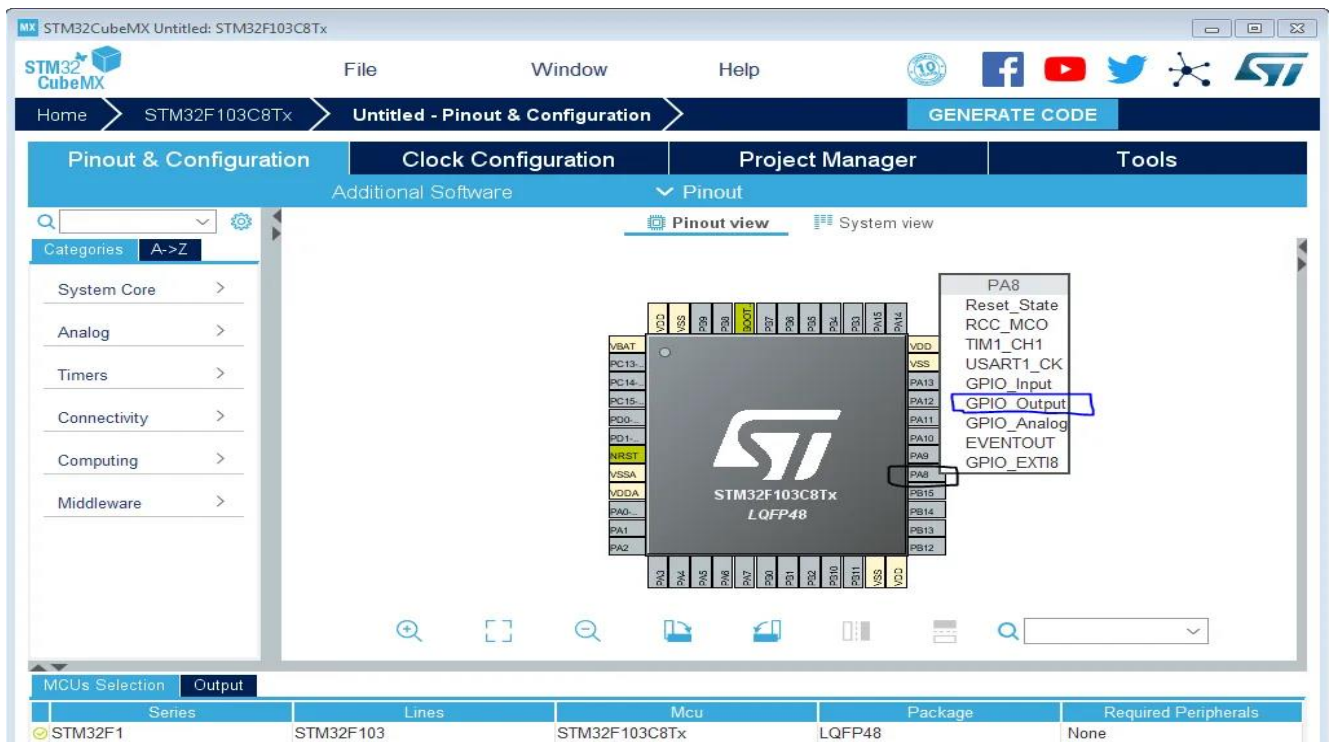


- **Step2: Choose The Target MCU & Double-Click Its Name**

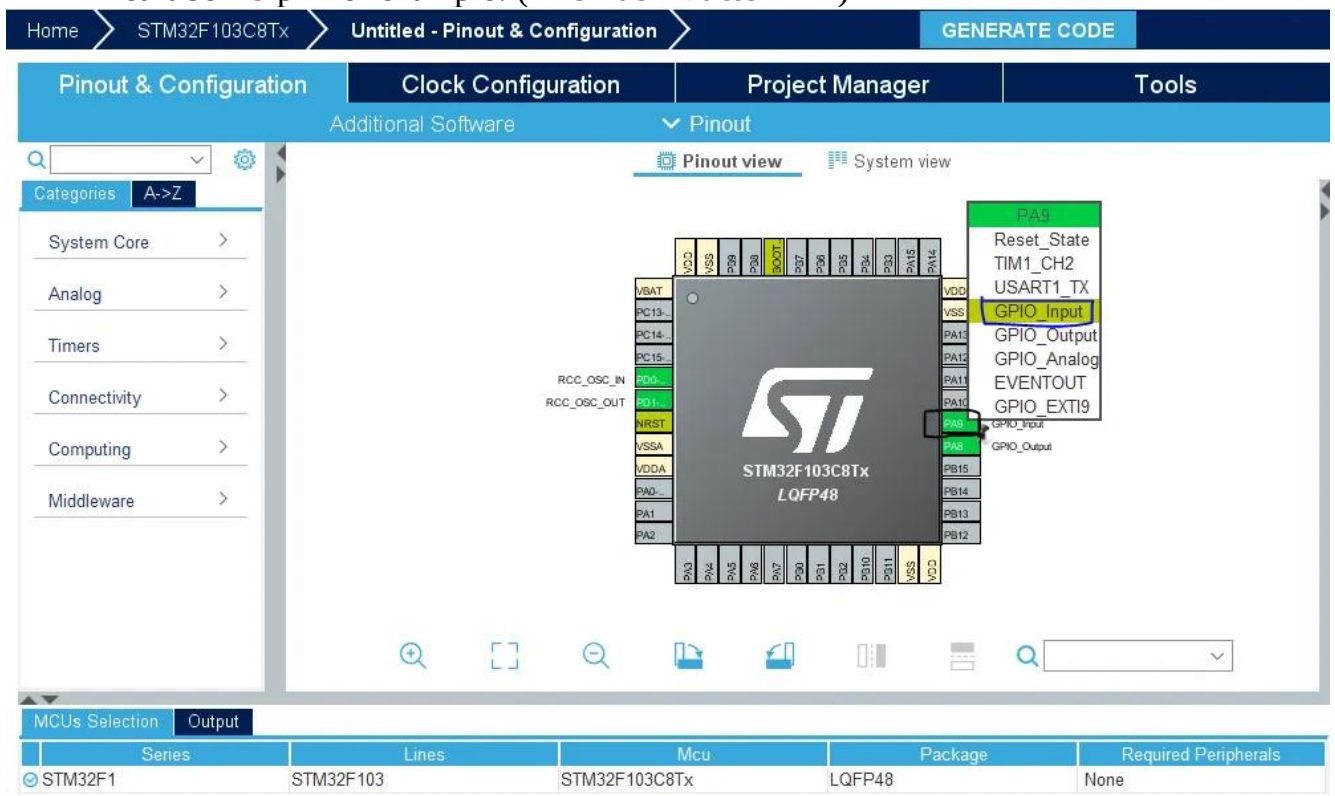


- **Step3: Click On The Pin You Want To Configure As An Output & Select Output Option**

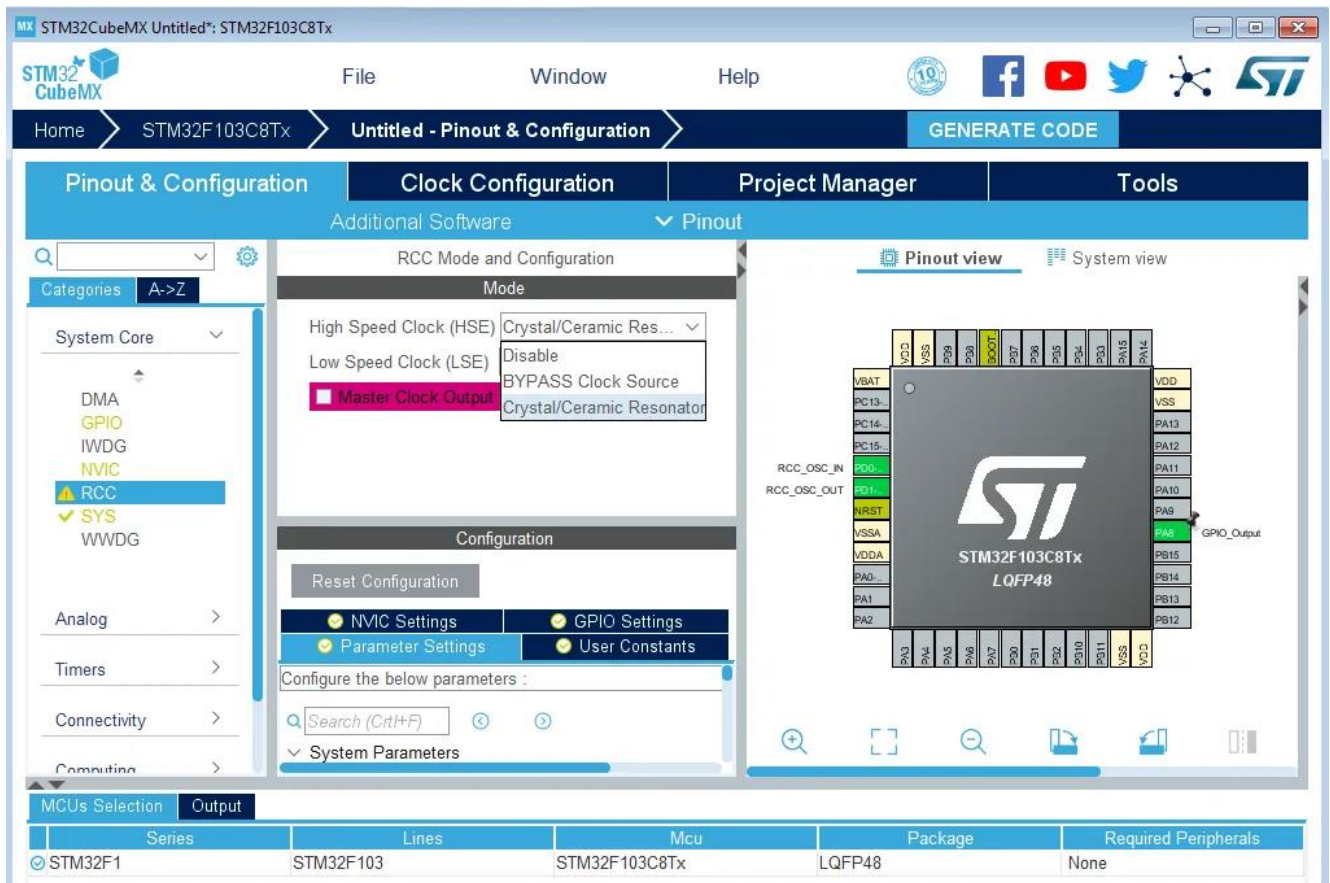
Let it be **A8** pin for example! (The LED Pin)



Step4: Click On The Pin You Want To Configure As An Input & Select Input Option
Let it be **A9** pin for example! (**The Push Button Pin**)

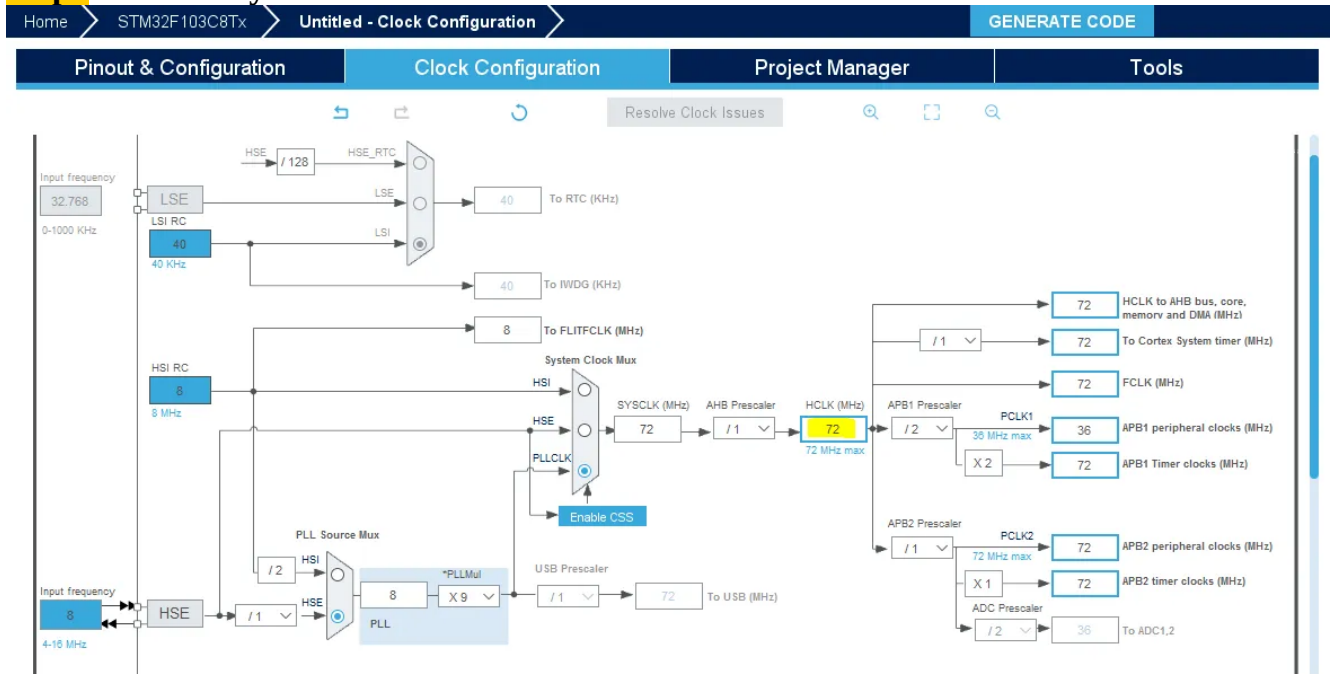


Step5: Set The RCC External Clock Source



Step6: Go To The Clock Configuration

Step7: Set The System Clock To Be 72MHz Or Whatever You Want



Step8: Name & Generate The Project Initialization Code For CubeIDE or The IDE You're Using

Home > STM32F103C8Tx > Untitled - Project Manager > **GENERATE CODE**

Pinout & Configuration | **Clock Configuration** | **Project Manager** | **Tools**

Project

Project Settings

Project Name: LED_Button

Project Location: C:\Users\Khaled Magdy\Desktop\ Browse

Application Structure: Basic ☐ Do not generate the ma...

Toolchain Folder Location: C:\Users\Khaled Magdy\Desktop\LED_Button\

Toolchain / IDE: STM32CubeIDE ☒ Generate Under ...

Code Generator

Advanced Settings

Linker Settings

Minimum Heap Size: 0x200

Minimum Stack Size: 0x400

Mcu and Firmware Package

Mcu Reference: STM32F103C8Tx

Firmware Package Name and Version:

MCUs Selection | Output

Series	Lines	Mcu	Package	Required Pen
STM32F1	STM32F103	STM32F103C8Tx	LQFP48	None

Then, open the project in the IDE you're using. And head over to the main.c file. So we can start writing the application code and have a look at the initialization code generated by the STM32 CubeMX tool.

18.3. The Application Code In Keil-MDK

- **Open project in Keil-MDK and write addition codes as follow**

The CubeMX tool has now generated the initialization C-Code for us. There is nothing special about it except the GPIO init function shown down below.

```
static void MX_GPIO_Init(void)
{
    GPIO_InitTypeDef GPIO_InitStruct = {0};
    /* GPIO Ports Clock Enable */
    __HAL_RCC_GPIOD_CLK_ENABLE();
    __HAL_RCC_GPIOA_CLK_ENABLE();
    /*Configure GPIO pin Output Level */
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8, GPIO_PIN_RESET);
    /*Configure GPIO pin : PA8 */
    GPIO_InitStruct.Pin = GPIO_PIN_8;
    GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
    GPIO_InitStruct.Pull = GPIO_NOPULL;
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
    HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);
    /*Configure GPIO pin : PA9 */
    GPIO_InitStruct.Pin = GPIO_PIN_9;
    GPIO_InitStruct.Mode = GPIO_MODE_INPUT;
    GPIO_InitStruct.Pull = GPIO_NOPULL;    // <----- This Option
    HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);
}
```

As you can see, the input pin (A9) is set to Hi-Z or High-Impedance mode, it's called no-pull (no pull-up or down). You can, however, set the pin to pull-up or pull-down depending on your preference.

We'll head over again to the HAL documentation and search for the GPIO chapter, where we'll find this listing for the available APIs. The APIs are hyperlinked in the documentation file, so you can click the name of the function to go directly to its detailed description.

IO operation functions

This section contains the following APIs:

- [HAL_GPIO_ReadPin\(\)](#)
- [HAL_GPIO_WritePin\(\)](#)
- [HAL_GPIO_TogglePin\(\)](#)
- [HAL_GPIO_LockPin\(\)](#)
- [HAL_GPIO_EXTI_IRQHandler\(\)](#)

So, let's take a closer look at the `GPIO_ReadPin()` function as we'll be using it as well.

HAL_GPIO_ReadPin

Function name	GPIO_PinState HAL_GPIO_ReadPin (GPIO_TypeDef * GPIOx, uint16_t GPIO_Pin)
Function description	Read the specified input port pin.
Parameters	• GPIOx: where x can be (A..H) to select the GPIO peripheral for STM32L4 family • GPIO_Pin: specifies the port bit to read. This parameter can be <code>GPIO_PIN_x</code> where x can be (0..15).
Return values	• The: input port pin value.

After reading the documentation and getting familiar with the available APIs, you are ready to go. In this LAB, we'll read the pin state for the push button (A9) and decide whether to drive the LED pin (A8) High or Low. It's a very simple application as you can tell by looking at the code down below.

```
int main(void)
{
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();
    while (1)
    {
        // IF Button Is Pressed
        if(HAL_GPIO_ReadPin (GPIOA, GPIO_PIN_9))
        {
            // Set The LED ON!
            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8, GPIO_PIN_SET);
        }
        else
        {
            // Else .. Turn LED OFF!
            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8, GPIO_PIN_RESET);
        }
    }
}
```

```
}
}
```

- Create Hex file
- Load Hex file into circuit in proteus
- Run

18.4. Pracetise

